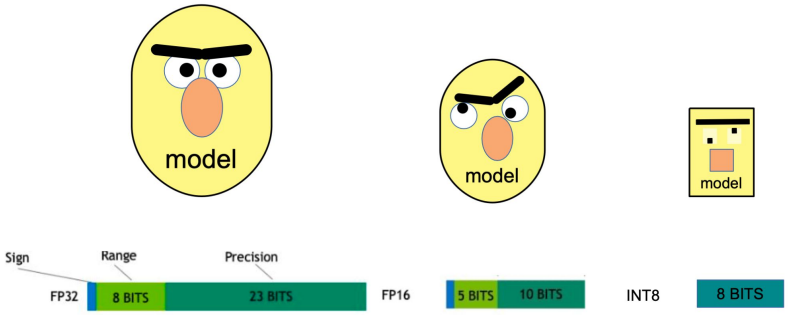
Transformers in Deep Learning

Lecture 8

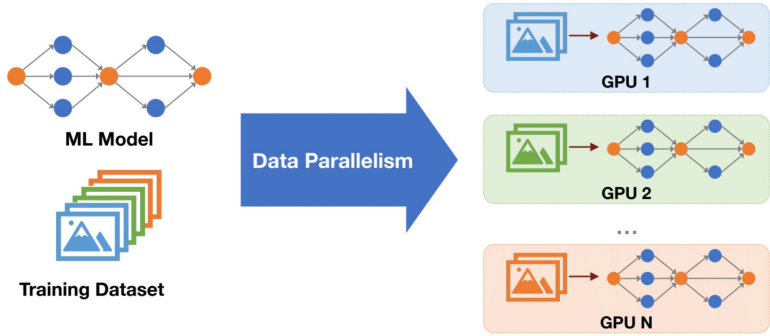
Week 8

Quantization



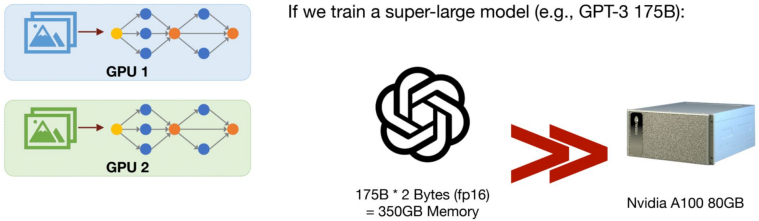
https://github.com/yandexdataschool/nlp_course/blob/2025

Data Parallelism



https://hanlab.mit.edu/courses/2024-fall-65940

ZeRO-1/2/3

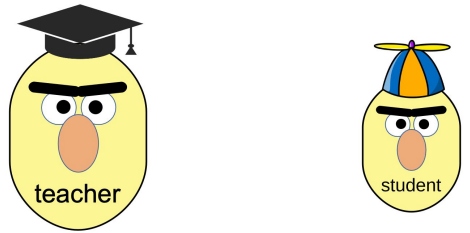


Even the best GPU **CANNOT** fit the model weights into memory!

Further, training requires to store gradients and optimizer states.

https://uvadlc-notebooks.readthedocs.io/en/latest/tutorial_notebooks/scaling/JAX/tensor_parallel_simple.html

Distillation: what if we need small domain-specific model?

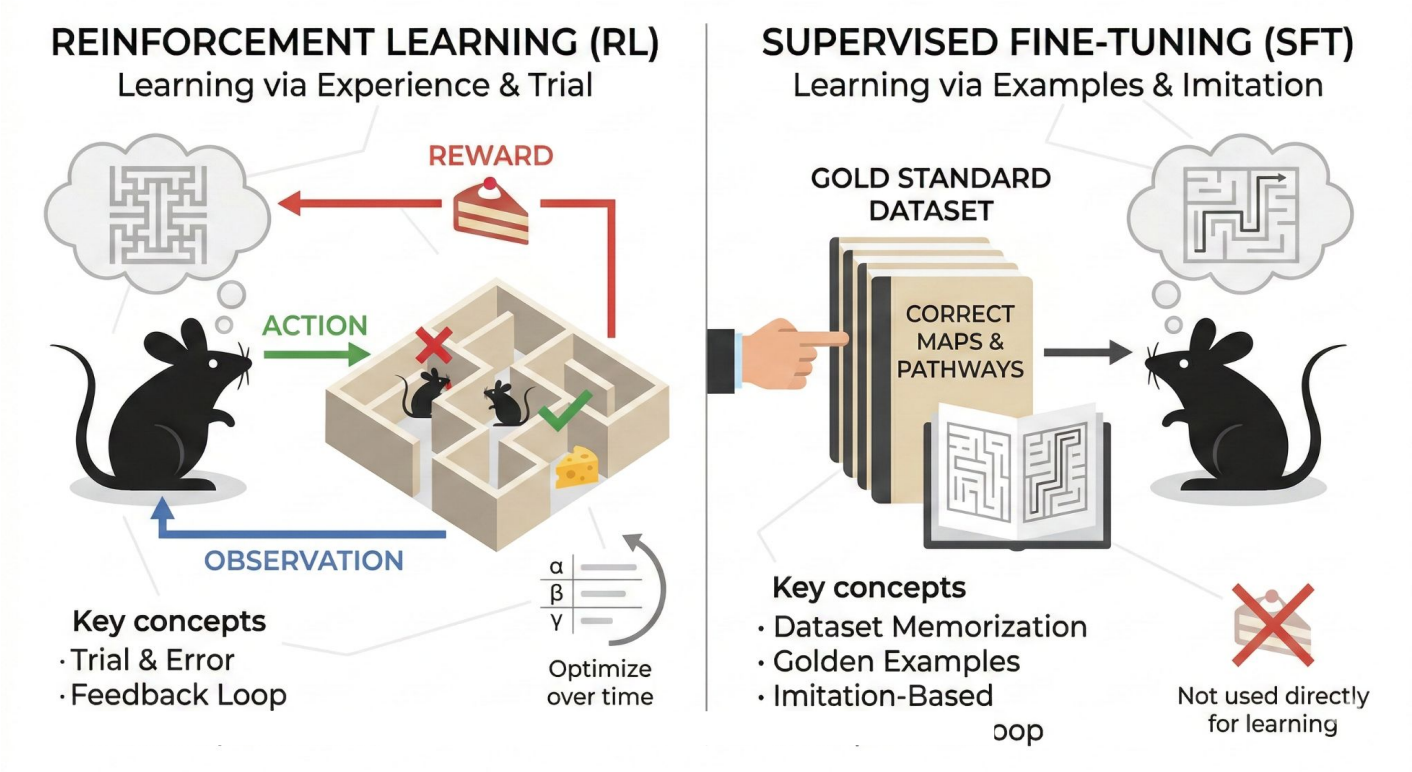


First, get the best performing model regardless of size

Then, train a more compact model to approximate it!

https://github.com/yandexdataschool/nlp_course/blob/2025

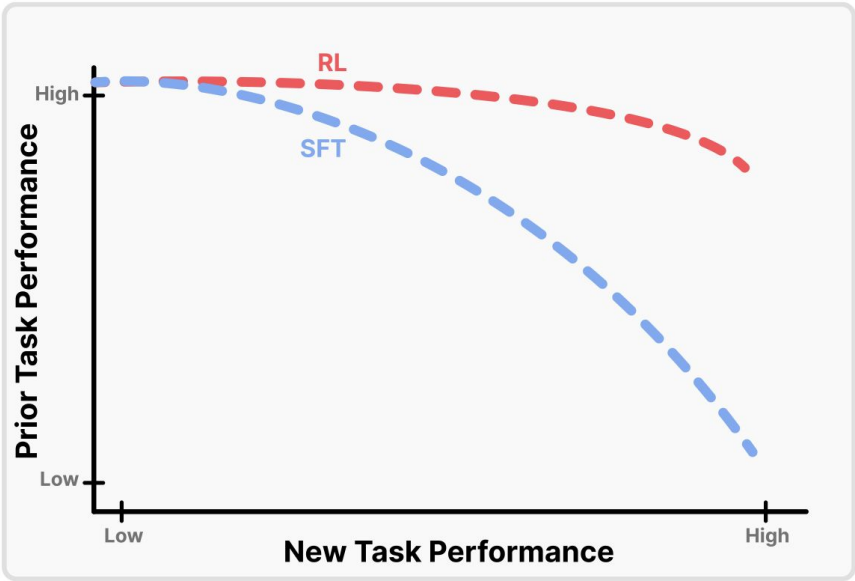
RL (Reinforcement Learning)



https://modal.com/blog/how-much-vram-need-fine-tuning?utm_source=chatgpt.com

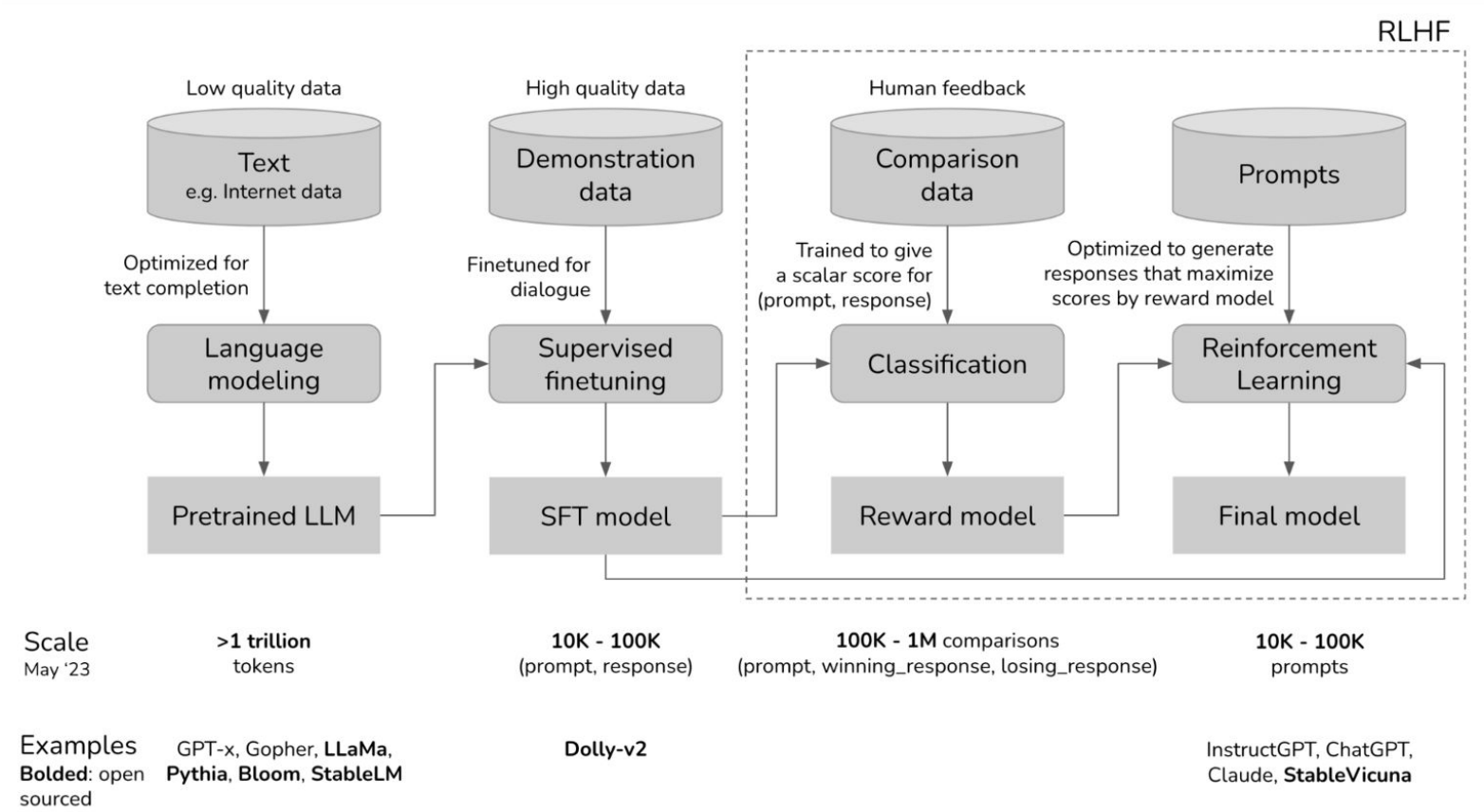
Why RL?

1) Learning from negative feedback
2) RL forgets less
3) Better Generalization
4) Better Exploration
5) Reasoning and Self-Correction



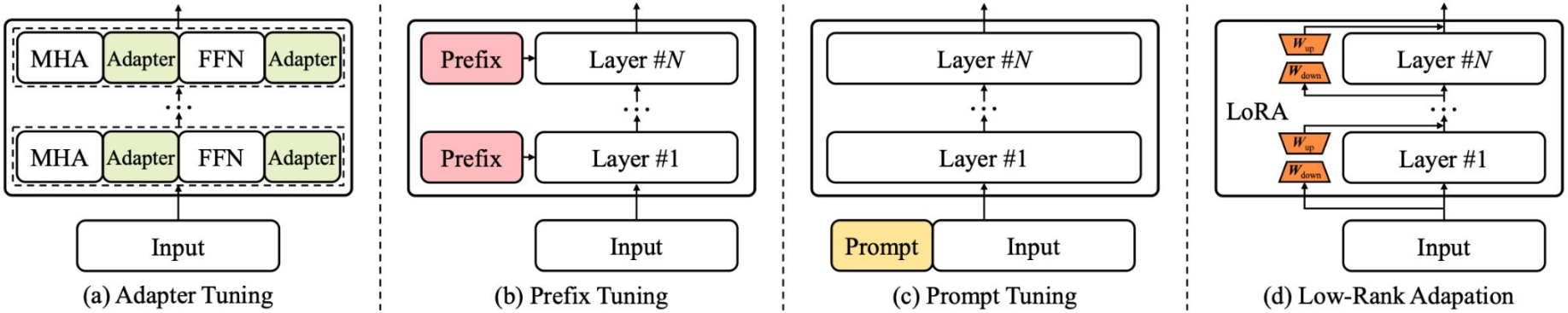
https://arxiv.org/abs2509.04259

RLHF (Preference Tuning)



https://arxiv.org/abs/2203.02155

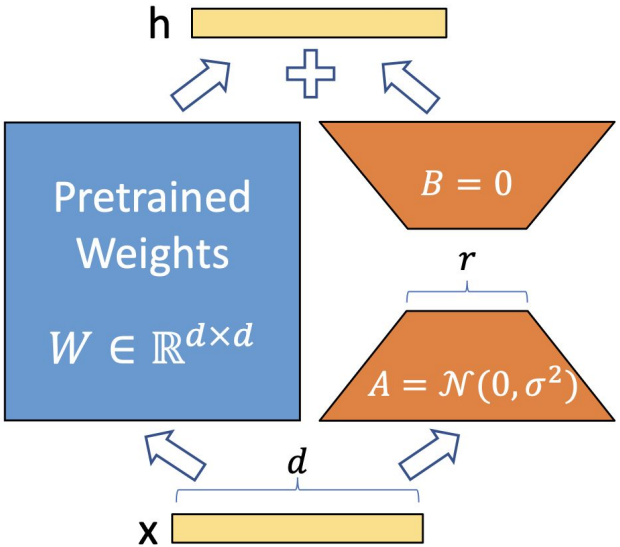
PEFT: Parameter Efficient Tuning

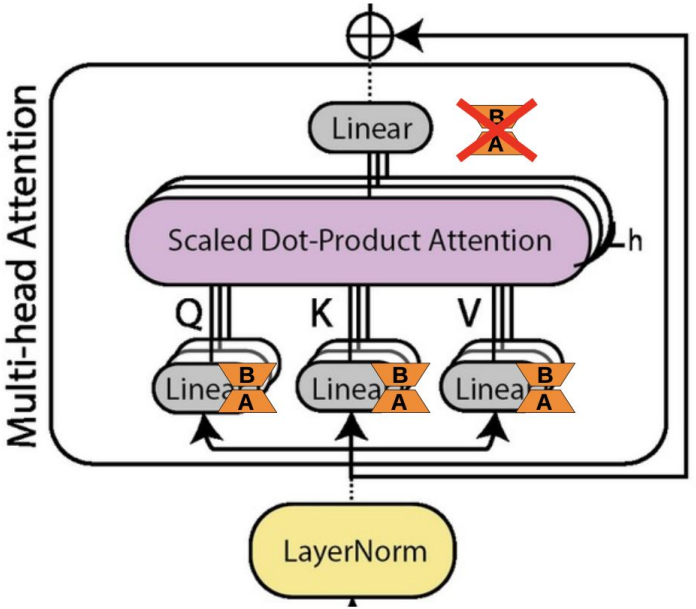


Let's train specific model parts instead of Full Transformer!

https://arxiv.org/abs/2303.18223

LoRA: add adapters in parallel

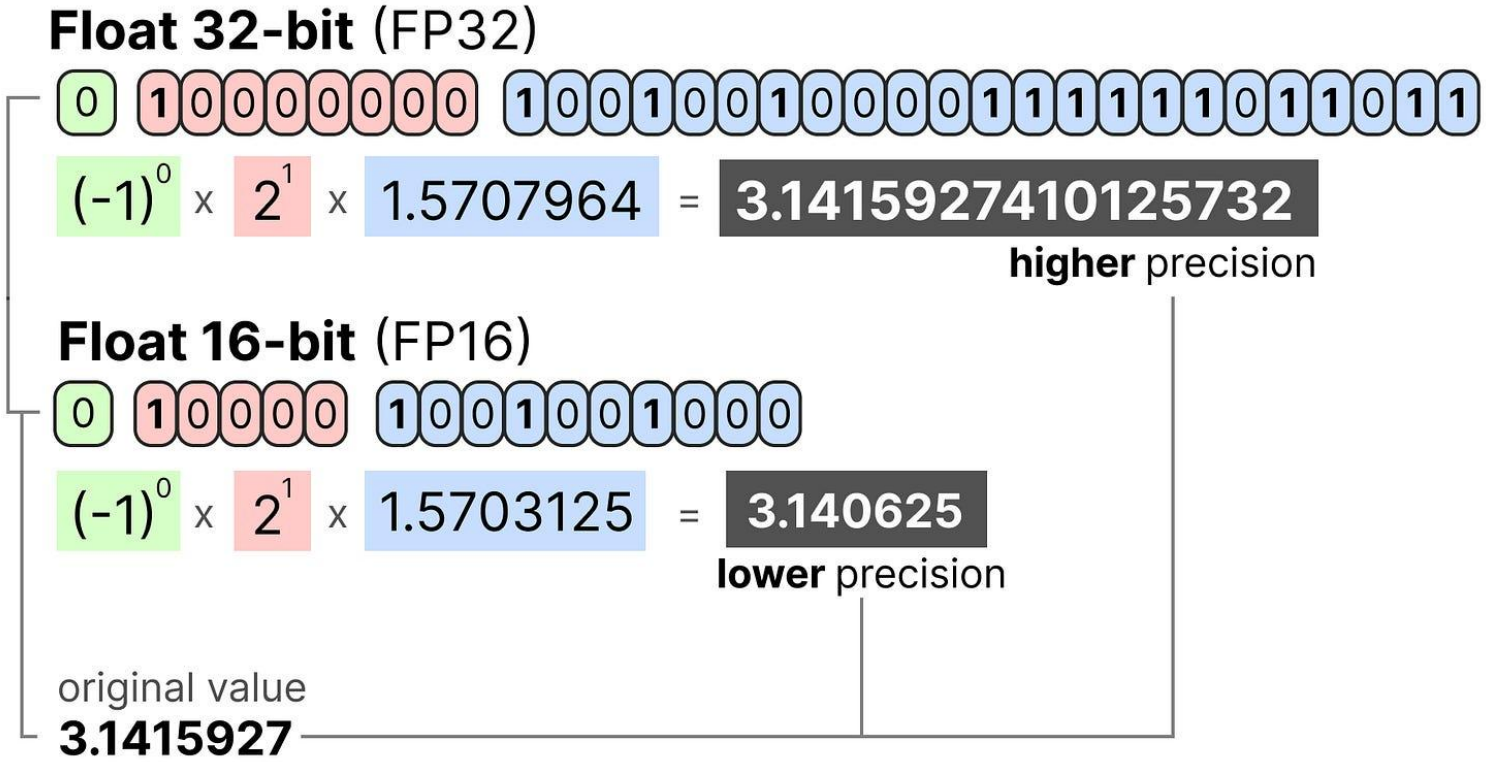




Add LoRA to all Linear Layers, but not to Embeddings, MoE, Logits

https://arxiv.org/abs/2106.09685

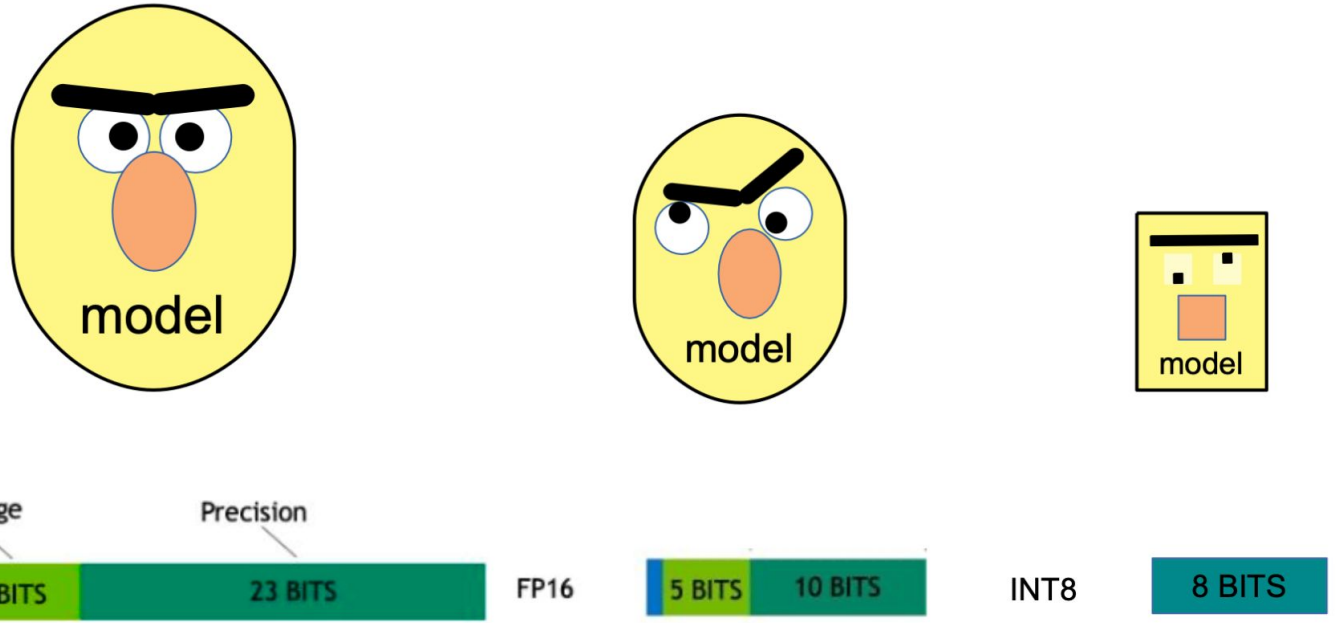
How computer store model weights?



https://www.maartengrootendorst.com/blog/quantization/

Quantization

Quantization

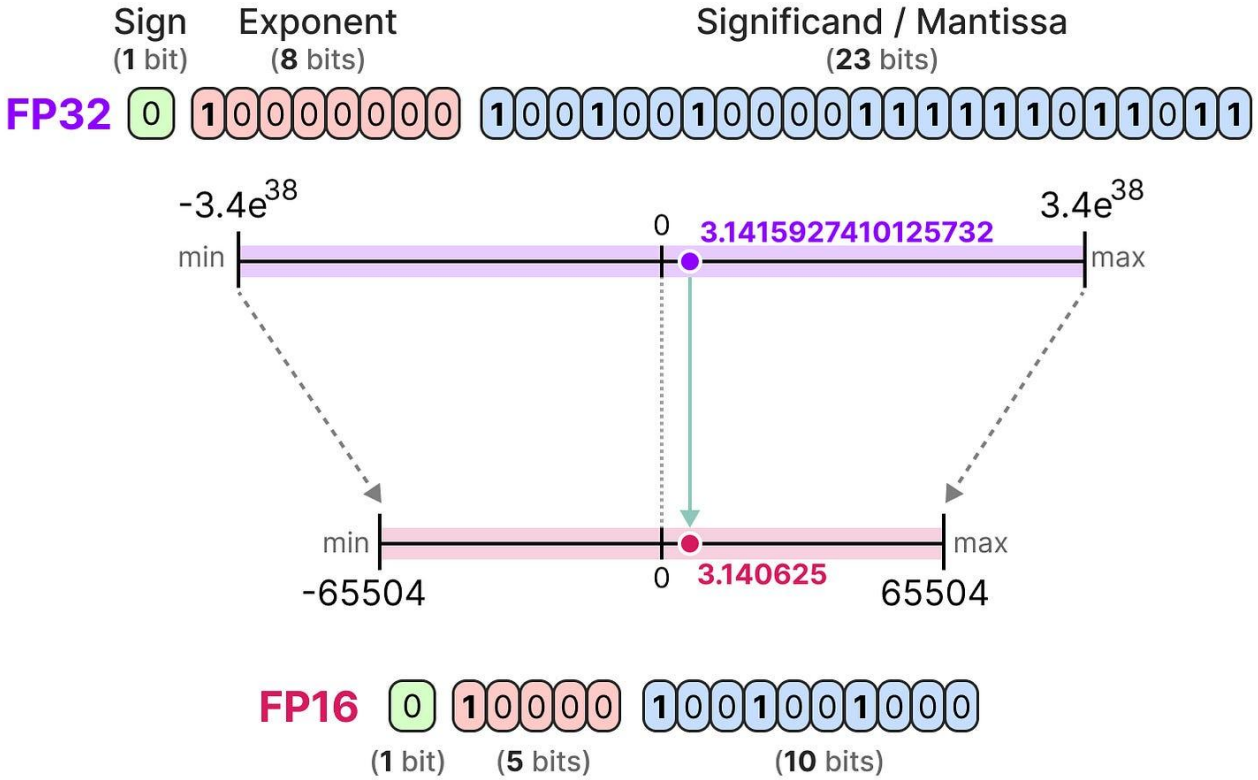


https://github.com/yandexdataschool/nlp_course/blob/2025

Quantization

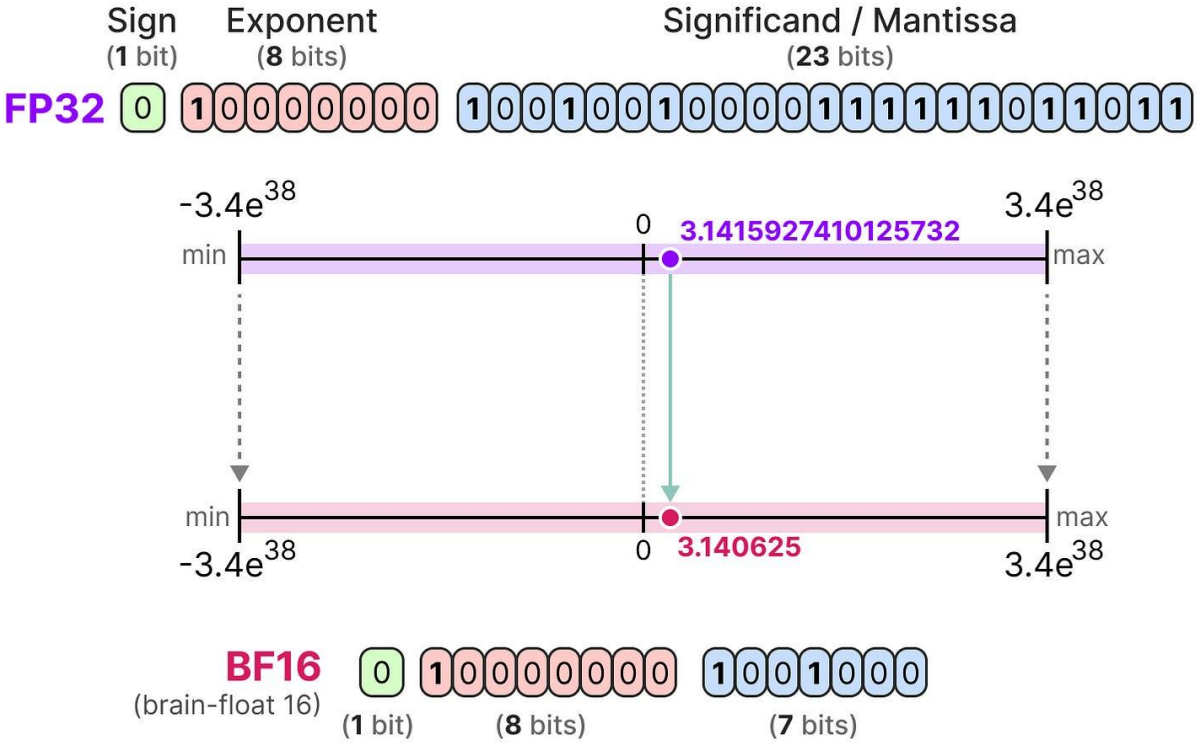
https://www.maartengrootendorst.com/blog/quantization/

Quantization: fp16



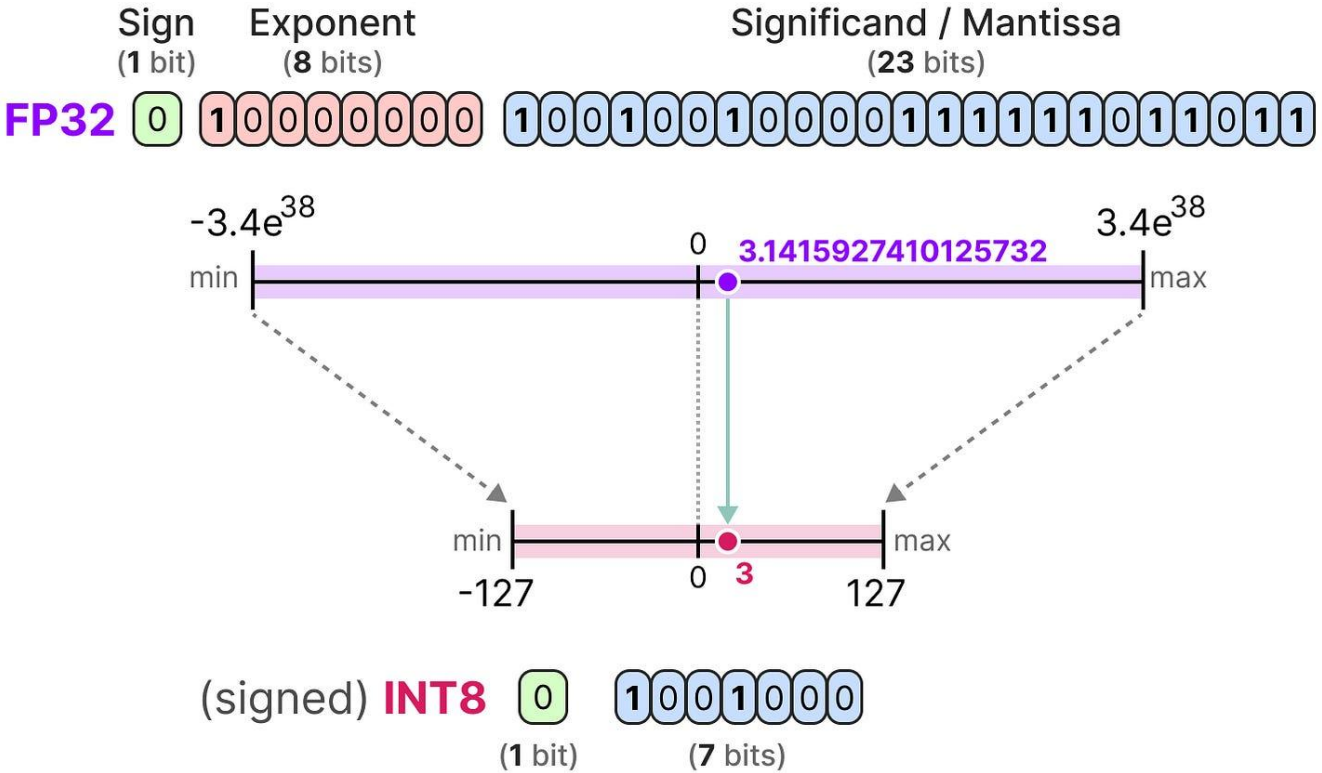
https://www.maartengrootendorst.com/blog/quantization/

Quantization: bfloat16 (bf16)



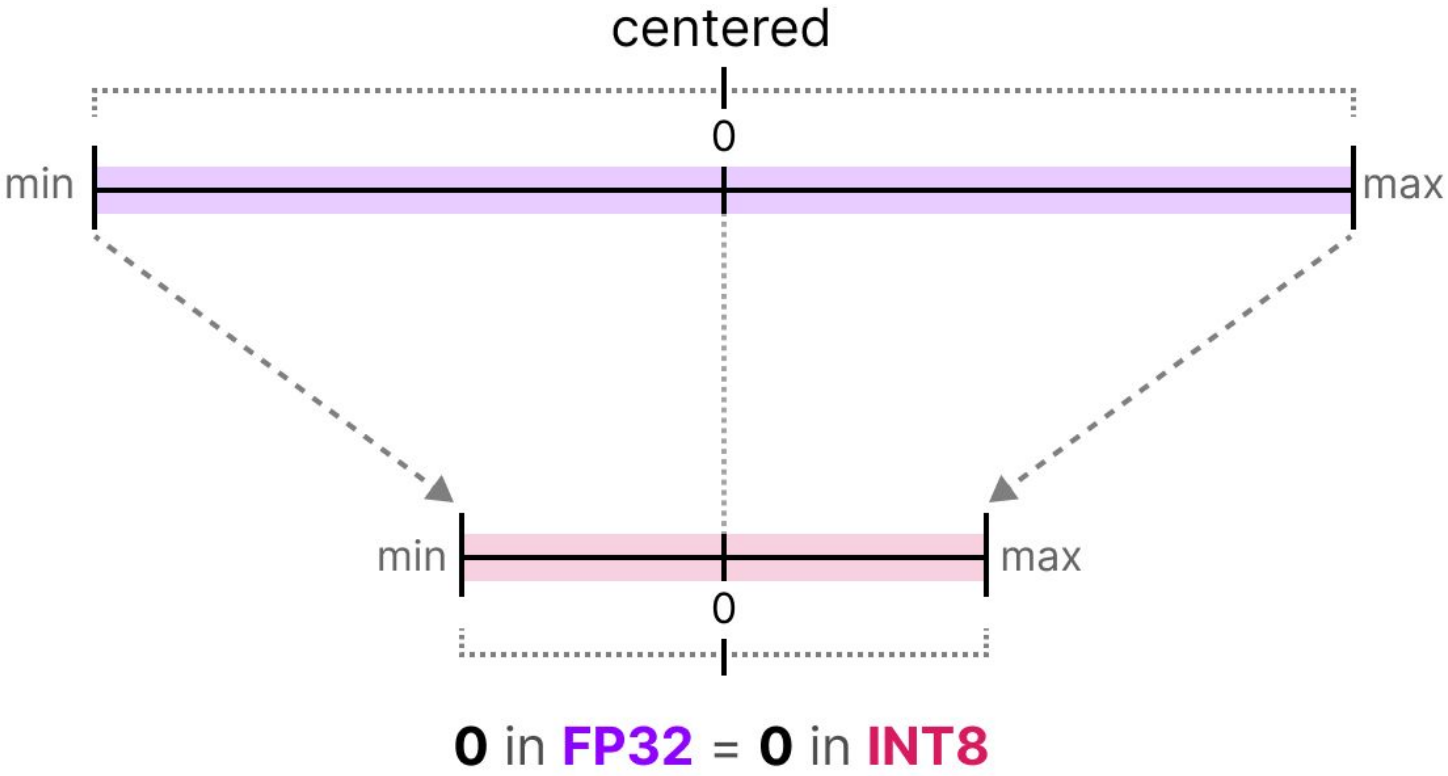
https://www.maartengrootendorst.com/blog/quantization/

Quantization: int8



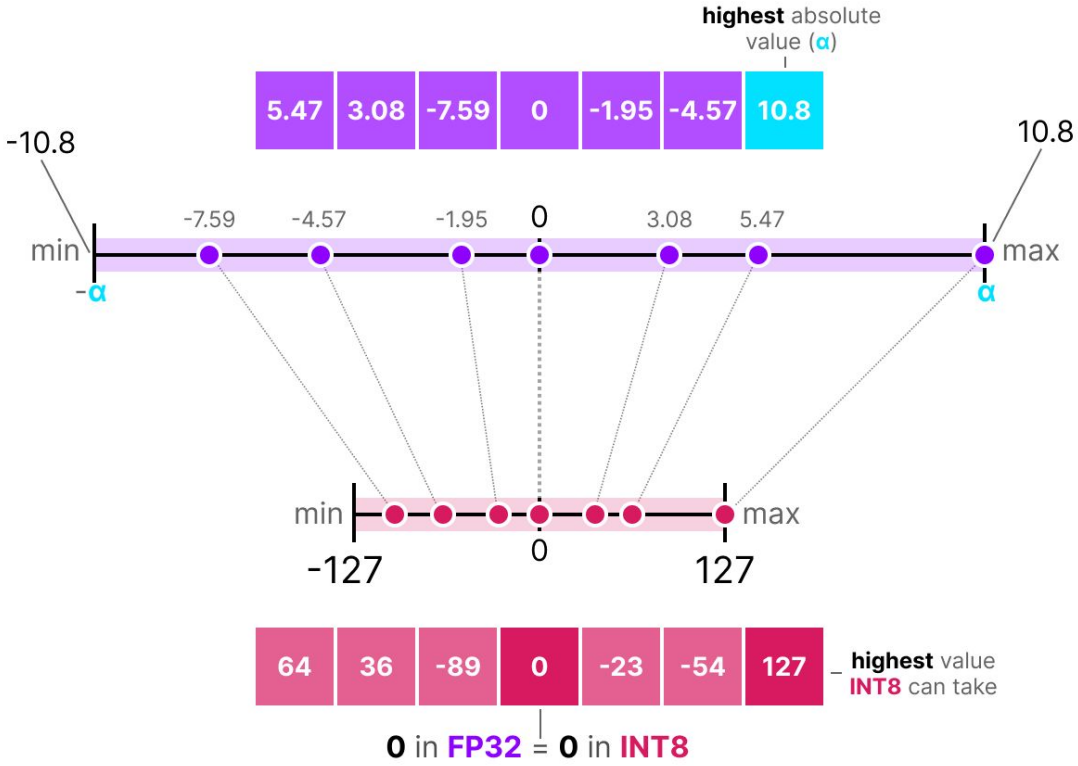
https://www.maartengrootendorst.com/blog/quantization/

Quantization: centered int8



https://www.maartengrootendorst.com/blog/quantization/

Quantization: centered int8



https://www.maartengrootendorst.com/blog/quantization/

Quantization: centered int8

$$s = \frac{2^{b-1}-1}{\alpha} \qquad \text{(scale factor)}$$

$$x_{quantized} = \text{round}\left(s \cdot x\right) \qquad \text{(quantization)}$$

Filling in the values would then give us the following:

$$s = \frac{127}{10.8} = 11.76 \qquad \text{(scale factor)}$$

$$x_{quantized} = \text{round}\left(11.76 \cdot \blacksquare\right) \qquad \text{(quantization)}$$

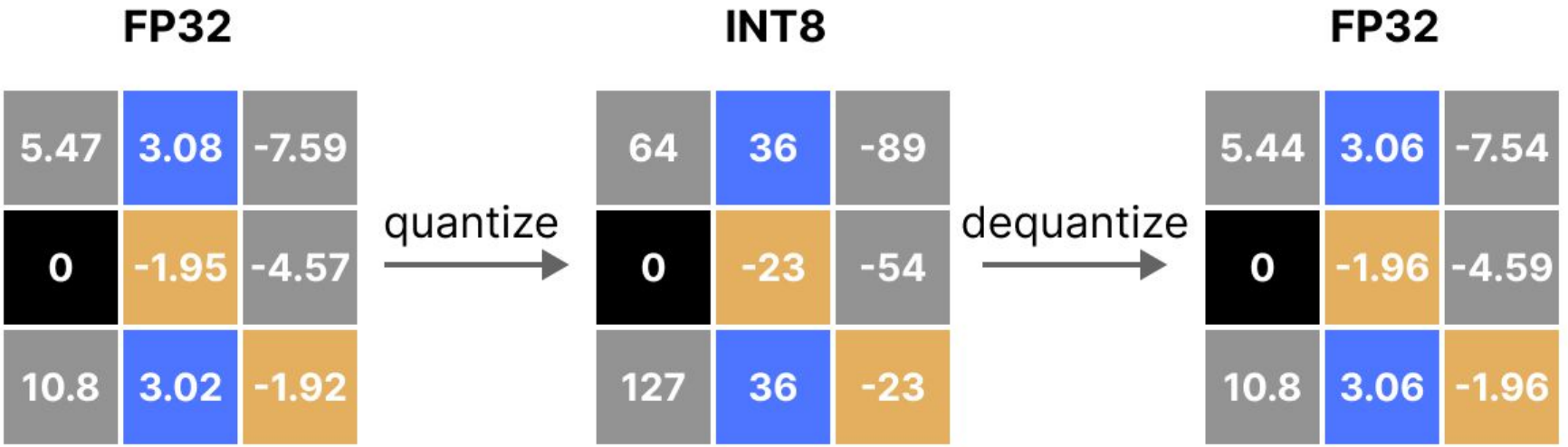
To retrieve the original FP32 values, we can use the previously calculated *scaling factor* (_ **s**) to *_dequantize* the quantized values.

$$x_{dequantized} = \frac{\blacksquare}{s} \qquad \text{(dequantize)}$$

- _ **b**_ is the number of bytes that we want to quantize to (8),
- **α** _ **** *is the _highest* absolute value,

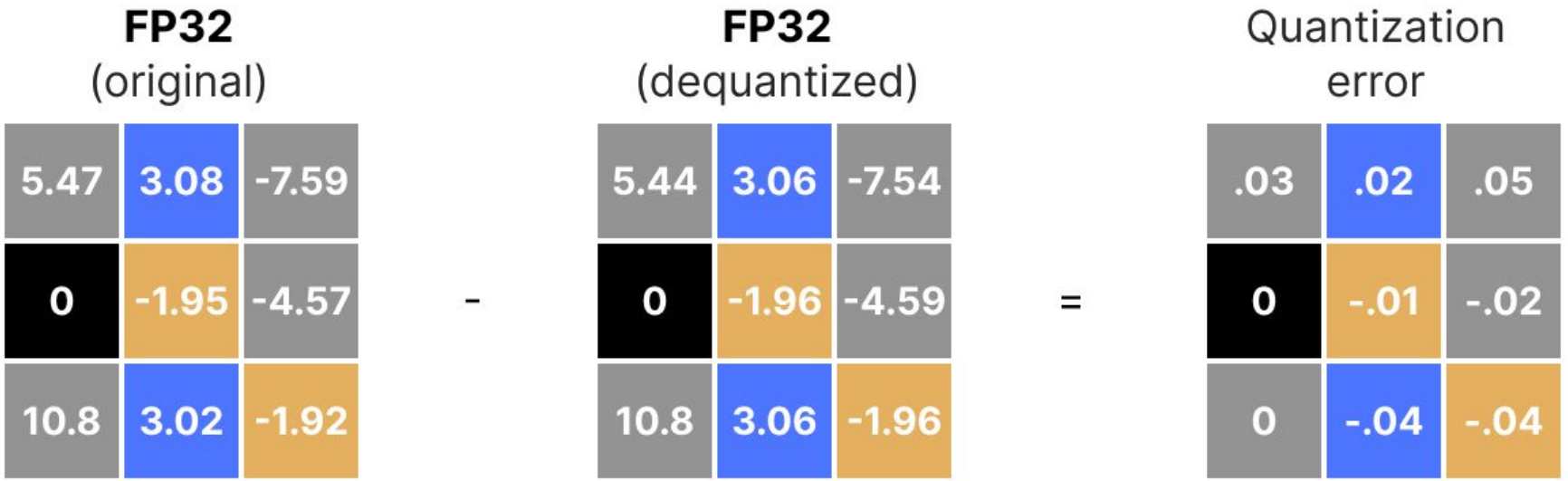
https://www.maartengrootendorst.com/blog/quantization/

Quantization: centered int8



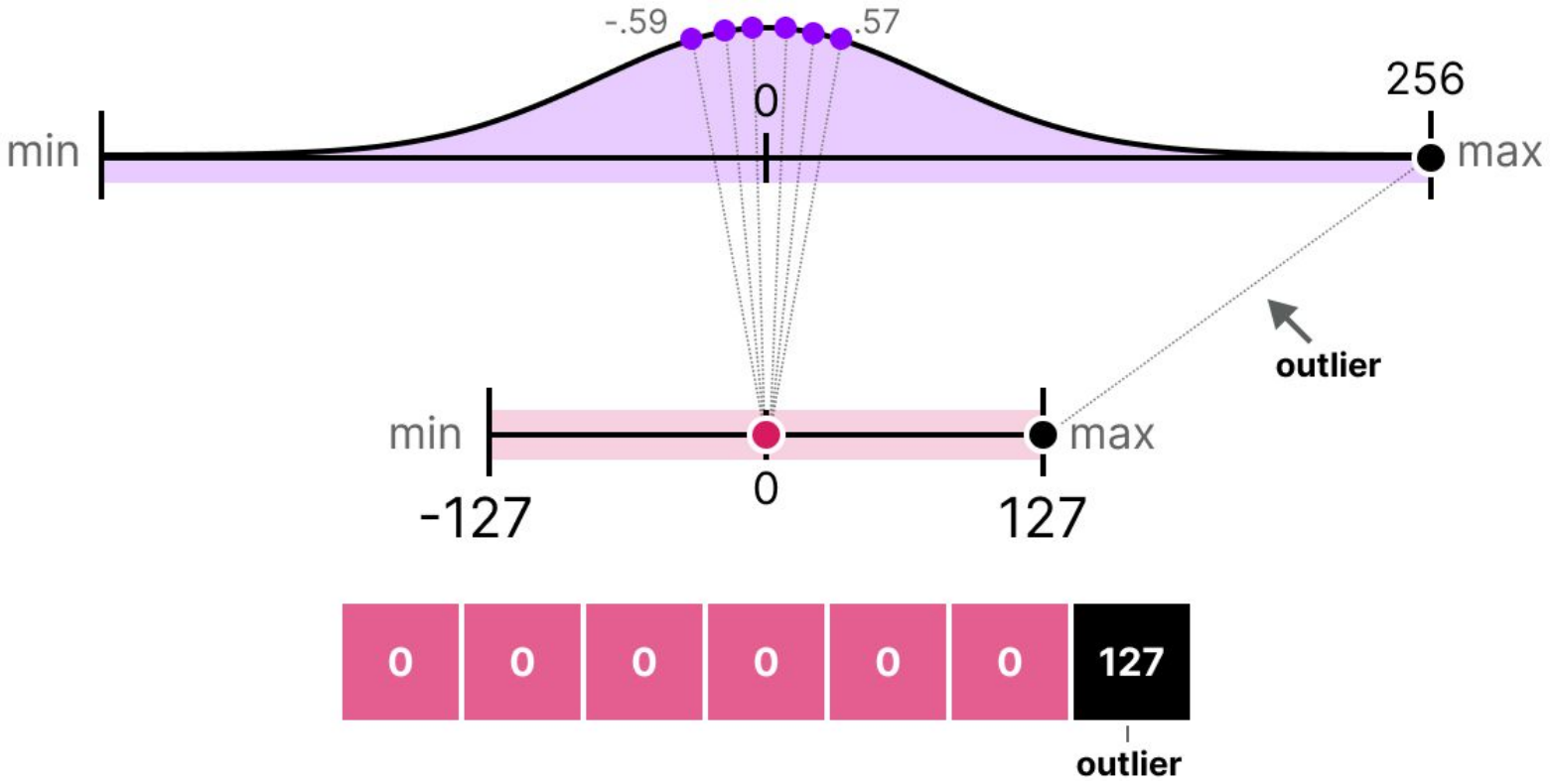
https://www.maartengrootendorst.com/blog/quantization/

Quantization: quantization error



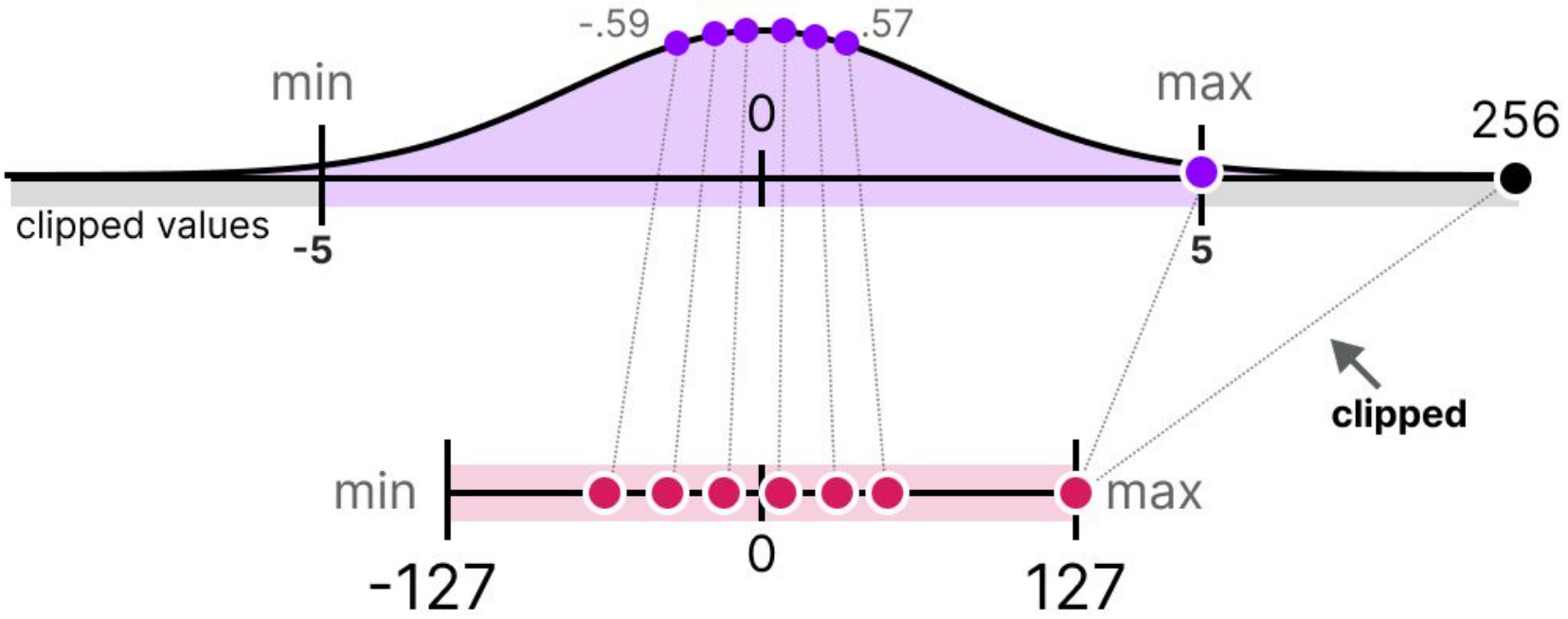
https://www.maartengrootendorst.com/blog/quantization/

Quantization: outliers



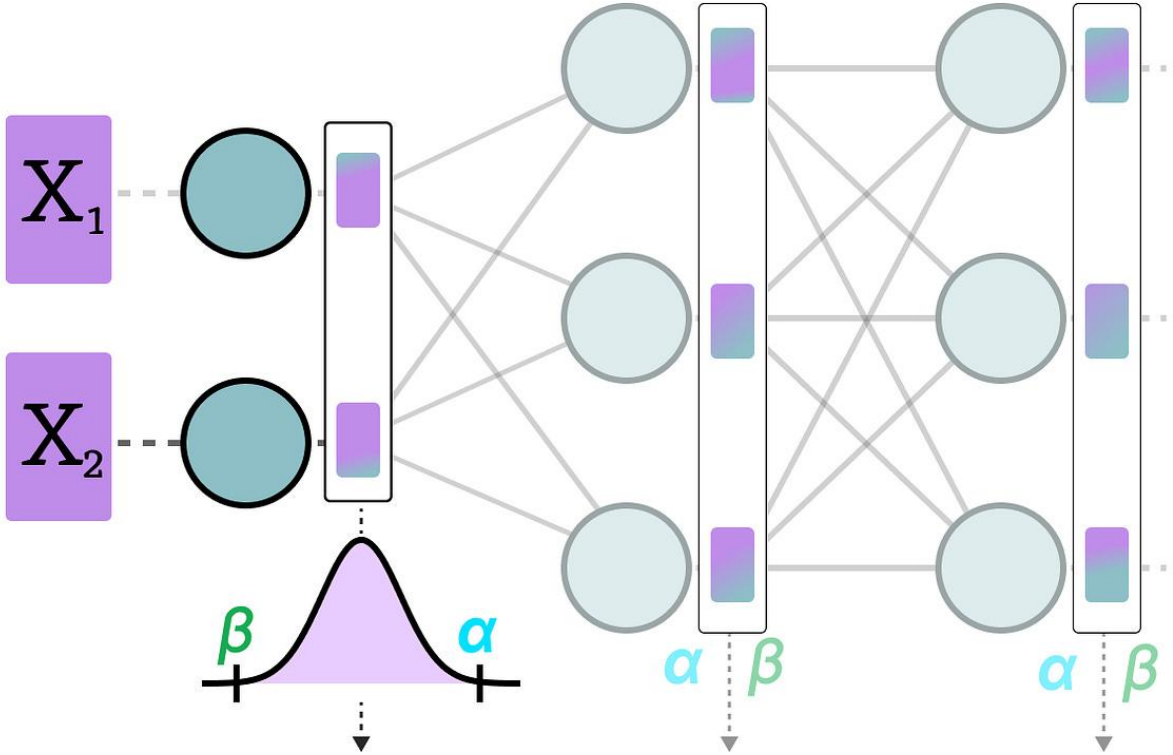
https://www.maartengrootendorst.com/blog/quantization/

Quantization: range clipping



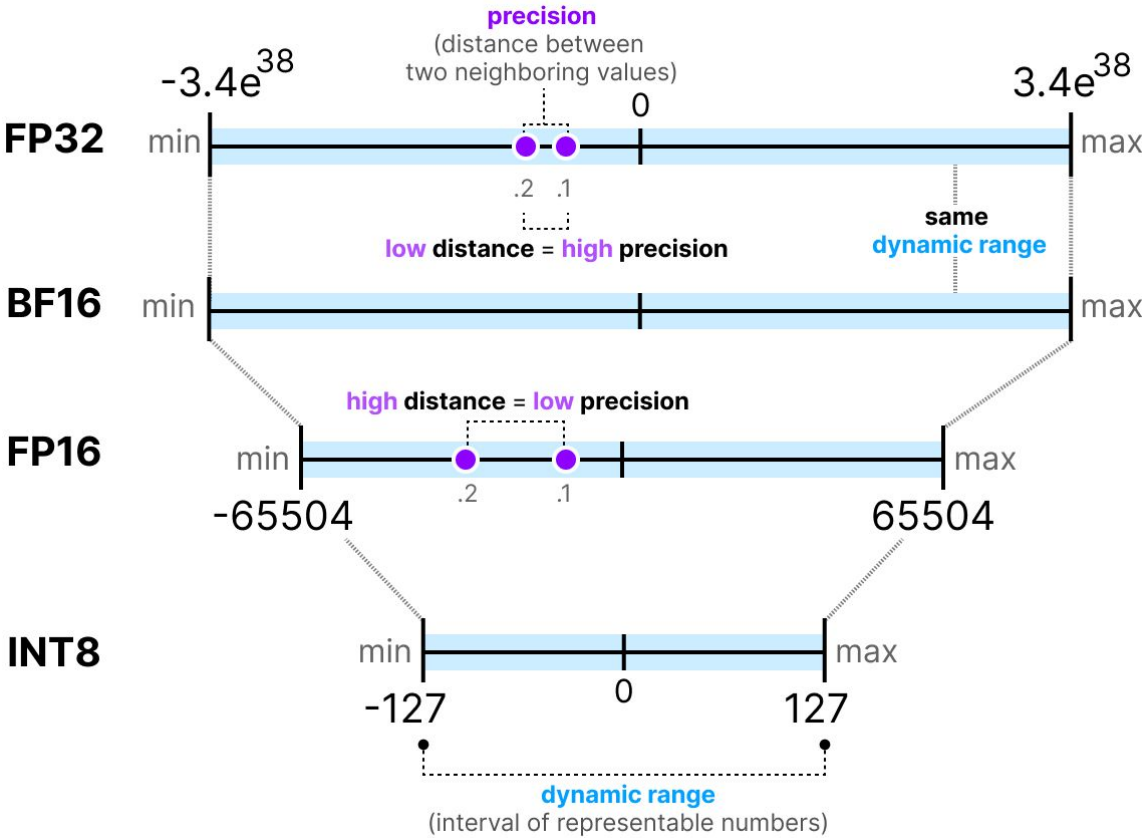
https://www.maartengrootendorst.com/blog/quantization/

Quantization: model forward



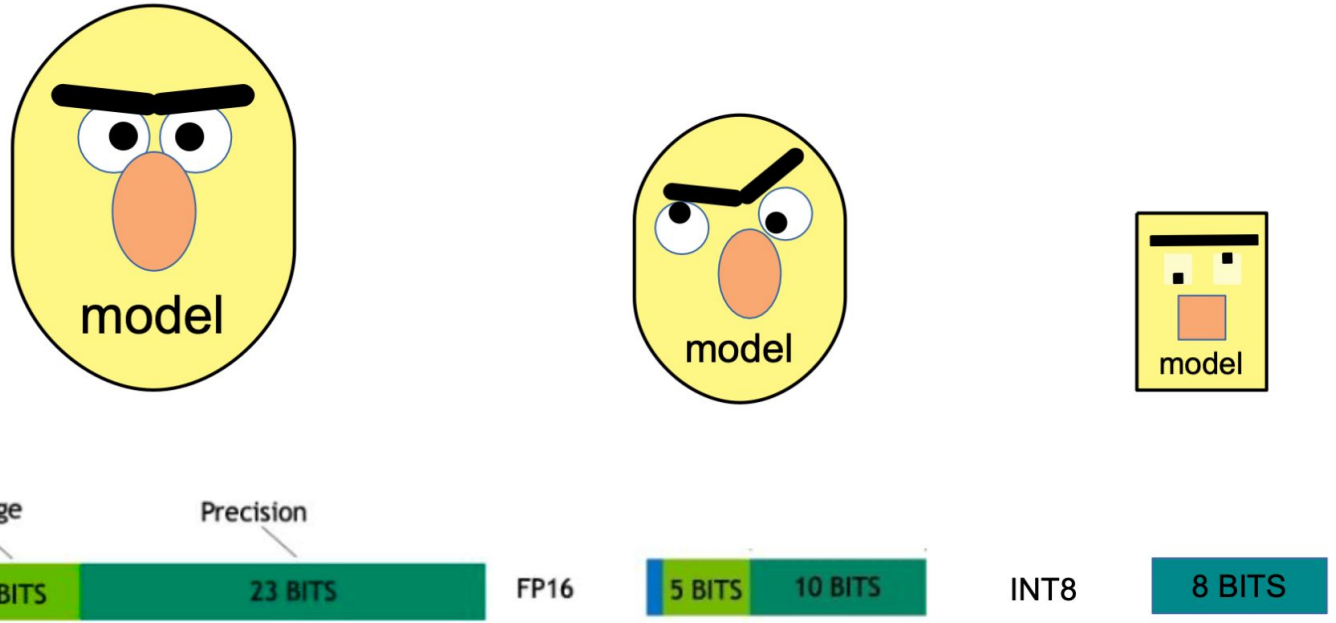
https://www.maartengrootendorst.com/blog/quantization/

Quantization



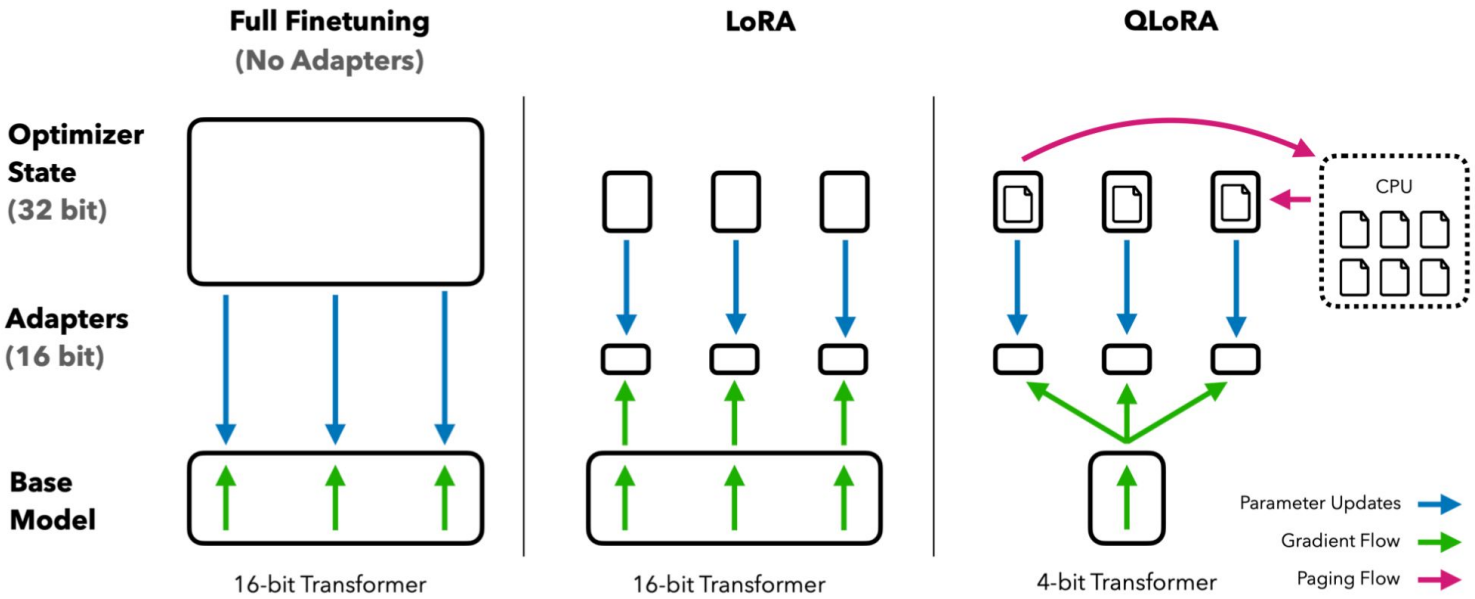
https://www.maartengrootendorst.com/blog/quantization/

Quantization



https://github.com/yandexdataschool/nlp_course/blob/2025

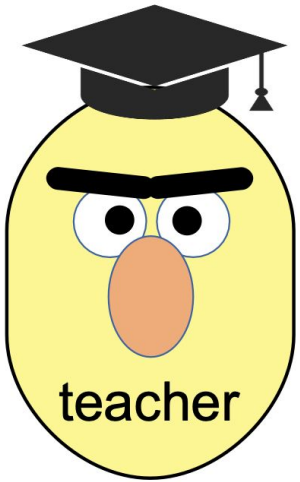
Quantized training: QLoRA



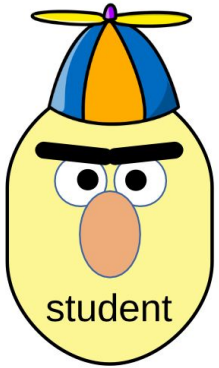
https://arxiv.org/pdf/2305.14314

Distillation

Distillation: what if we need small domain-specific model?



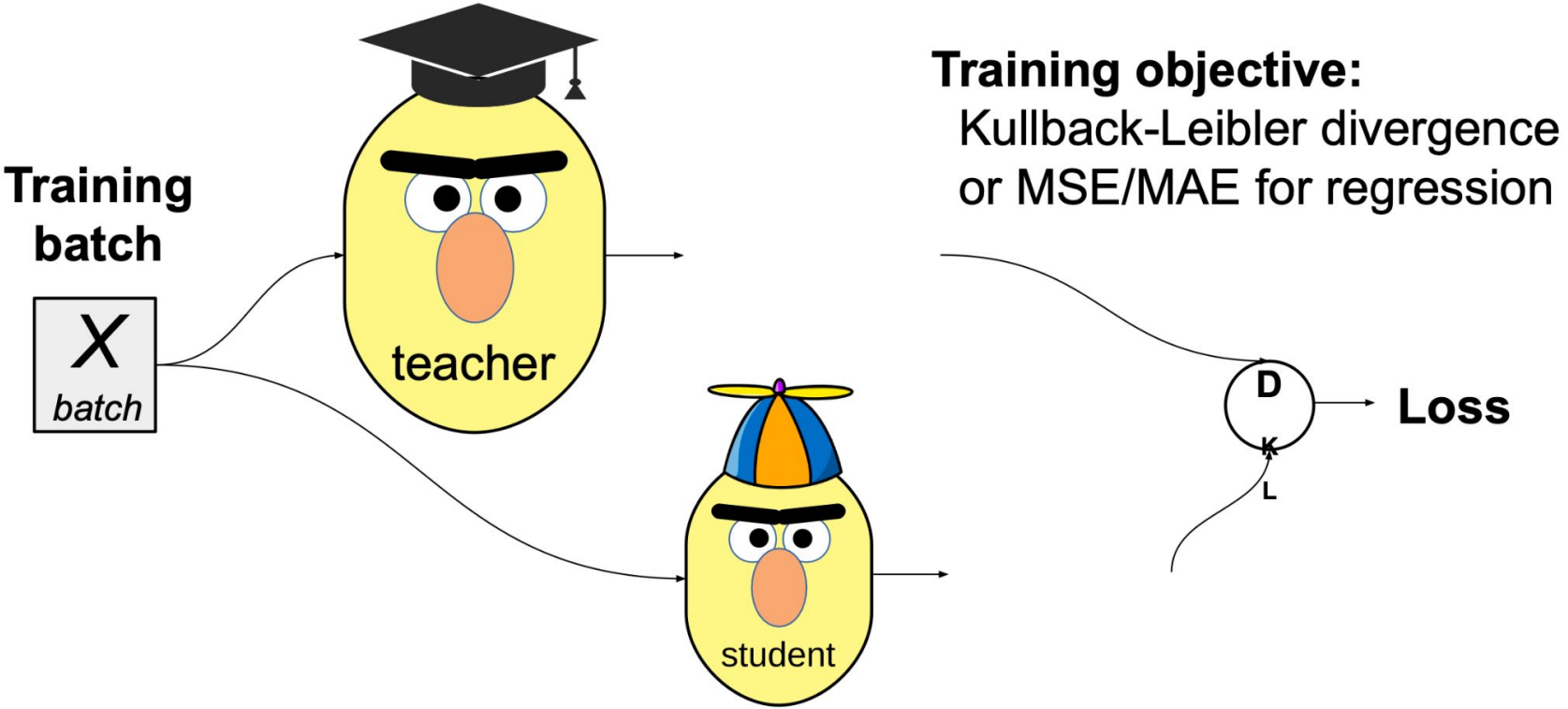
First, get the best performing model regardless of size



Then, train a more compact model to approximate it!

https://github.com/yandexdataschool/nlp_course/blob/2025

Distillation: what if we need small domain-specific model?



https://github.com/yandexdataschool/nlp_course/blob/2025

Distillation: student architecture choice

Naive: same but smaller, less layers / hidden units

- e.g. DistillBERT: https://arxiv.org/pdf/1910.01108.pdf
- Same as BERT-base, but with
- half as many layers
- (and ≈1.5 times faster)

Table 1: **DistilBERT retains 97% of BERT performance.** Comparison on the dev sets of the GLUE benchmark. ELMo results as reported by the authors. BERT and DistilBERT results are the medians of 5 runs with different seeds.

Model	**Score**	CoLA	MNLI	MRPC	QNLI	QQP	RTE	SST-2	STS-B	WNLI
ELMo	68.7	44.1	68.6	76.6	71.1	86.2	53.4	91.5	70.4	56.3
BERT-base	79.5	56.3	86.7	88.6	91.8	89.6	69.3	92.7	89.0	53.5
DistilBERT	77.0	51.3	82.2	87.5	89.2	88.5	59.9	91.3	86.9	56.3

Distillation: when it worth it?

- LLMs are trained up to compute budget
- Training the teacher (larger) model is VERY expensive
- Makes sense where:
  - Data is scarce, compute is not
  - A single teacher trains multiple students
  - Teacher achieves generalization and smaller model doesn’t

Read more:

- Distillation Scaling Laws
- Puzzle: Distillation-Based NAS for Inference-Optimized LLMs

https://github.com/yandexdataschool/nlp_course/blob/2025

Distributed GPU Training

Large models take longer Time to Train

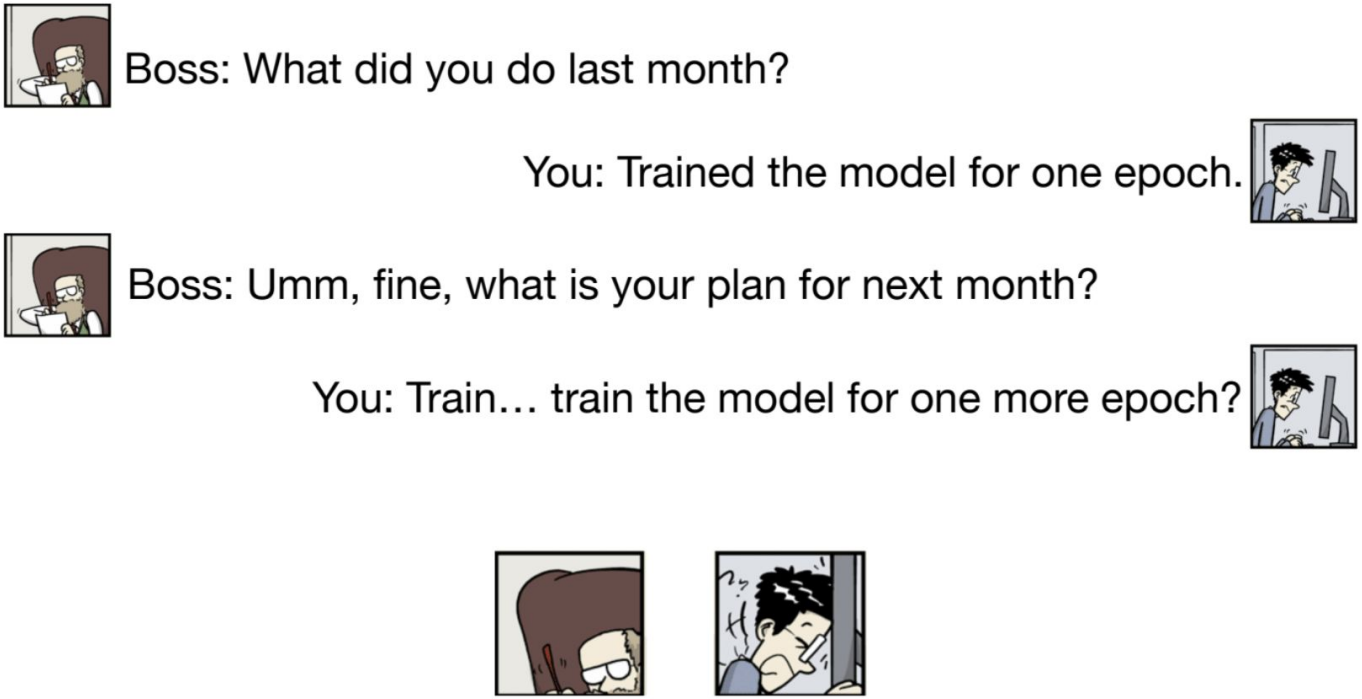
Models	#Params (M)	Training Time (GPU Hours)
ResNet-50	26	31
ResNet-101	45	44
BERT-Base	108	84
Turing-NLG 17B	17,000	TBA
GPT-3 175B	175,000	3,100,000

Measured on Nvidia A100

If without distributed training, a single GPU would take **355 years** to finish GPT-3!

https://hanlab.mit.edu/courses/2024-fall-65940

Large models take longer Time to Train



https://hanlab.mit.edu/courses/2024-fall-65940

Large models take longer Time to Train

- Developers / Researchers' time **are more valuable** than hardware .
- If a training takes **10 GPU days**
  - Parallelize with distributed training
  - 1024 GPUs can finish in 14 minutes (ideally)!
- The develop and research cycle will be greatly boosted

Let's see a use case of distributed training!

https://hanlab.mit.edu/courses/2024-fall-65940

SUMMIT Supercomputer

- CPU: 2 x 16 Core IBM POWER9 (connected via dual NVLINK bricks, 25GB/s each side)
- GPU: 6 x NVIDIA Tesla V100
- RAM: 512 GB DDR4 memory
- Storage: 250 PB
- Connection: Dual-rail EDR InfiniBand network of 200 Gbit/s

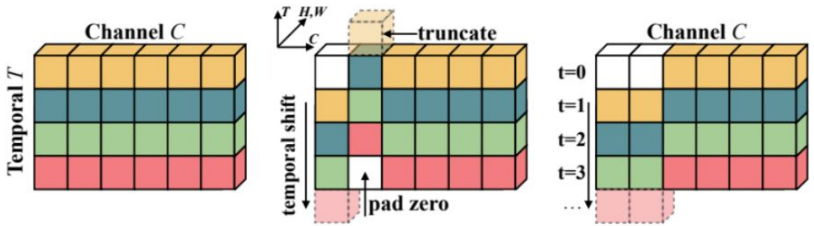
https://hanlab.mit.edu/courses/2024-fall-65940

Train Video model on 660 hours of Video

Models	Training Time	Accuracy
1 SUMMIT Nodes (6 GPUs)	49h 50min	74.1%
128 SUMMIT Nodes (768 GPUs)	28min	74.1%
256 SUMMIT Nodes (1536 GPUs)	14min (**211x**)	74.0%

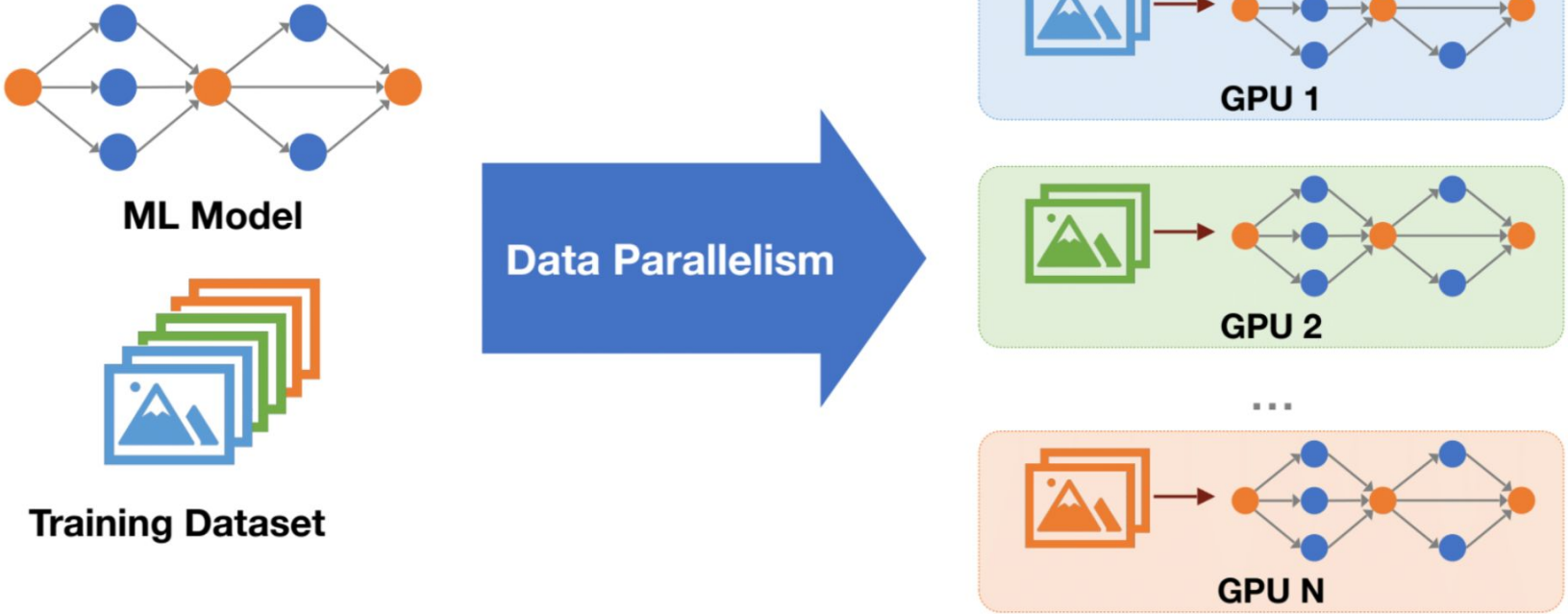
Speedup the training by 200x, **from 2 days to 14minutes.**

- **Model setup**: 8-frame ResNet-50 TSM for video recognition
- **Dataset**: Kinetics (240k training videos) x 100 epoch



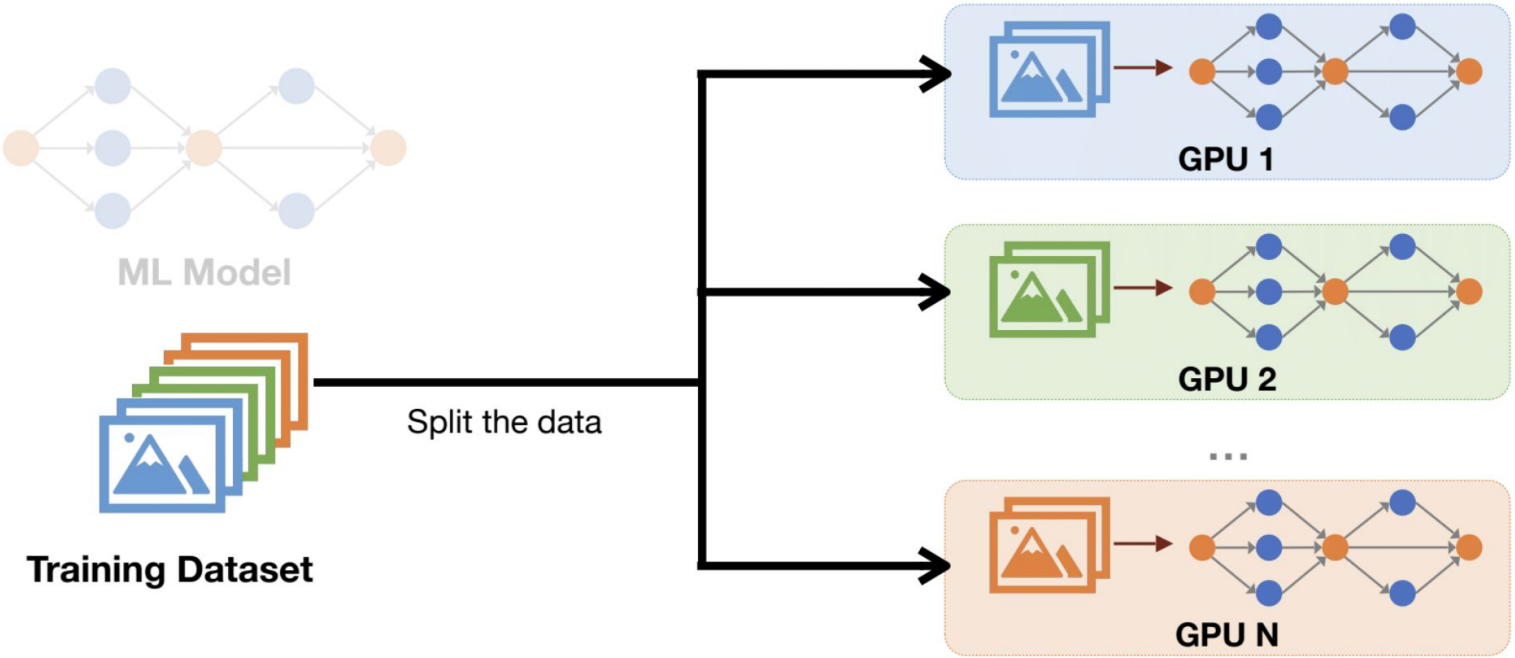
https://hanlab.mit.edu/courses/2024-fall-65940

Data Parallelism



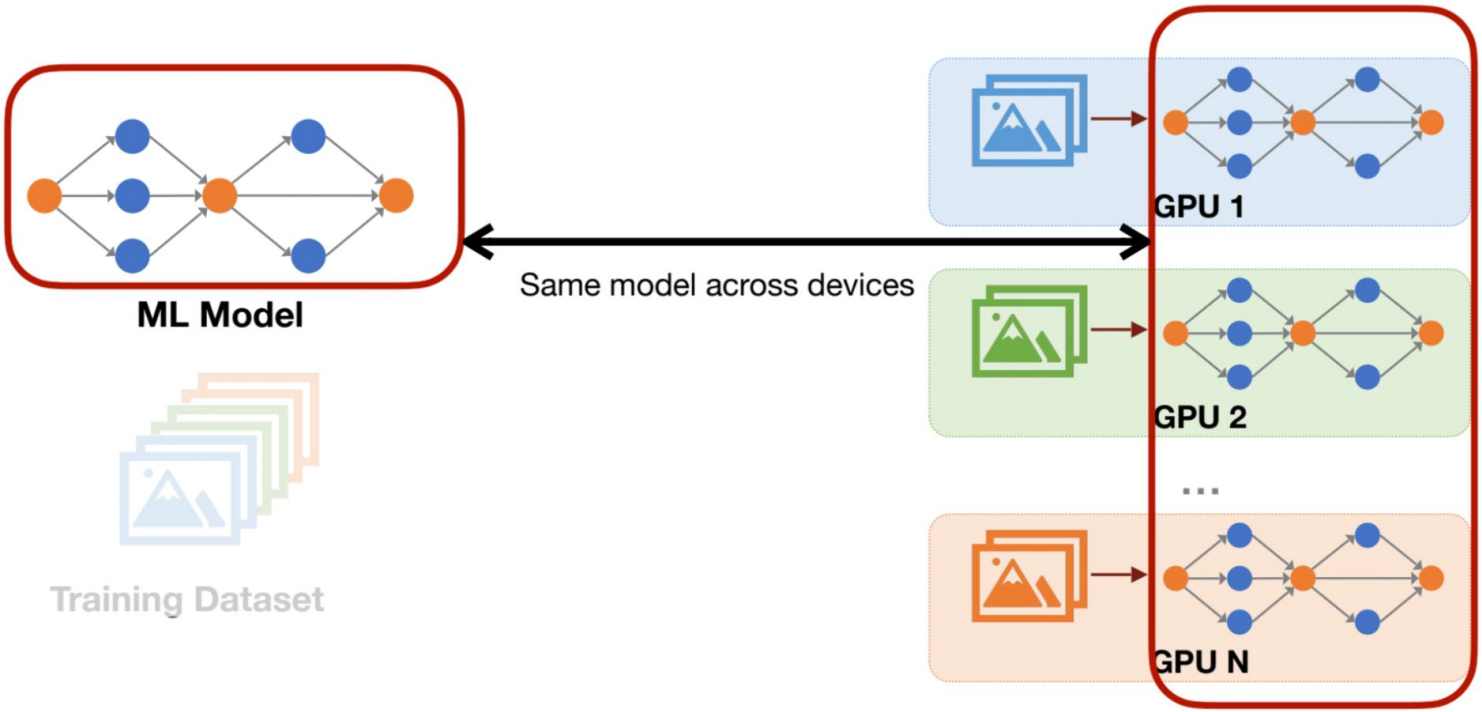
https://hanlab.mit.edu/courses/2024-fall-65940

Data Parallelism



https://hanlab.mit.edu/courses/2024-fall-65940

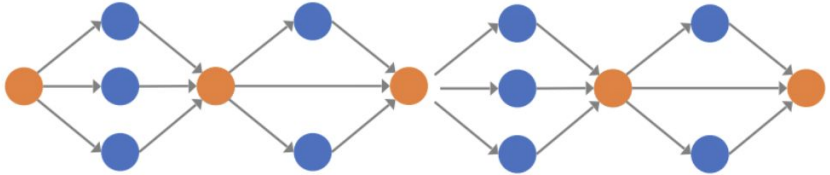
Data Parallelism



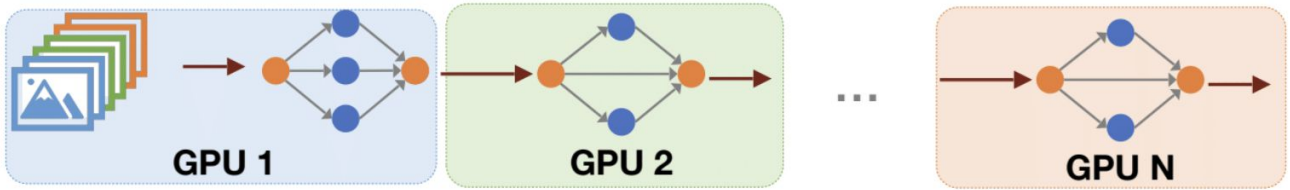
https://hanlab.mit.edu/courses/2024-fall-65940

Pipline Parallelism

Training Dataset

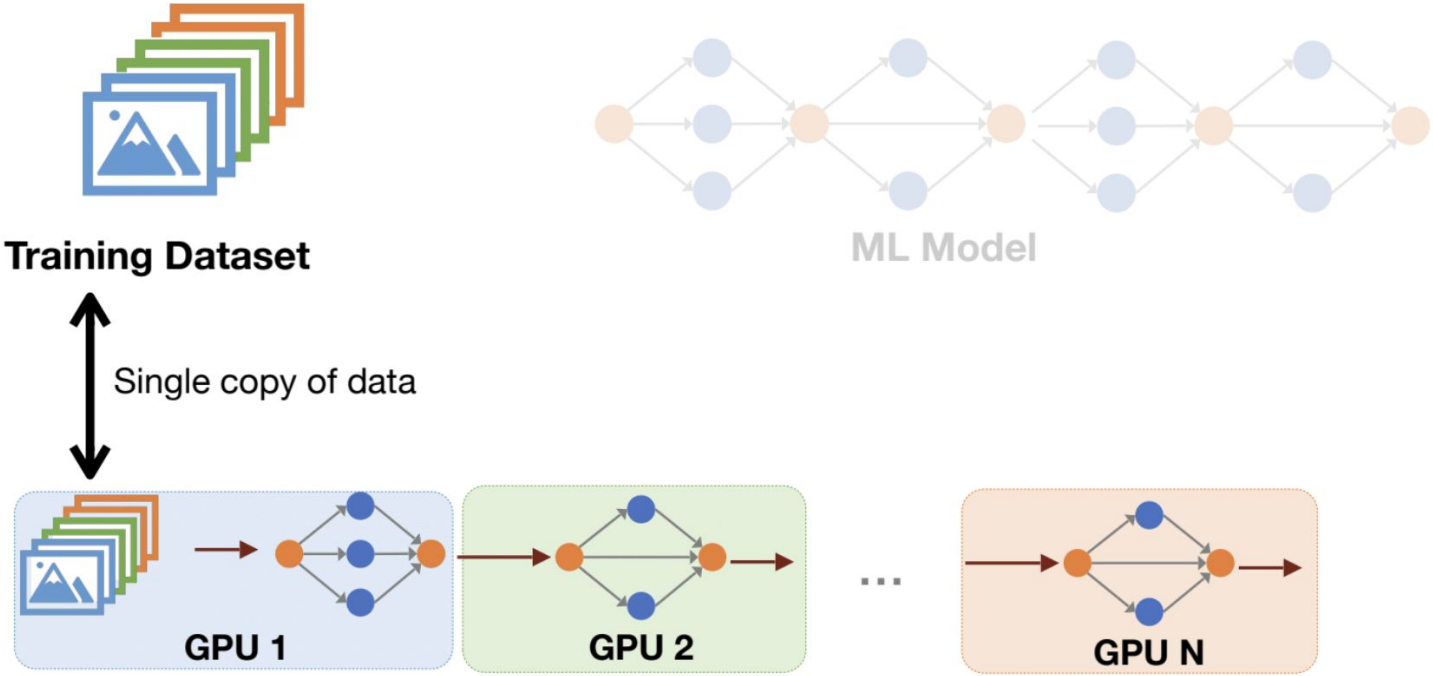


ML Model



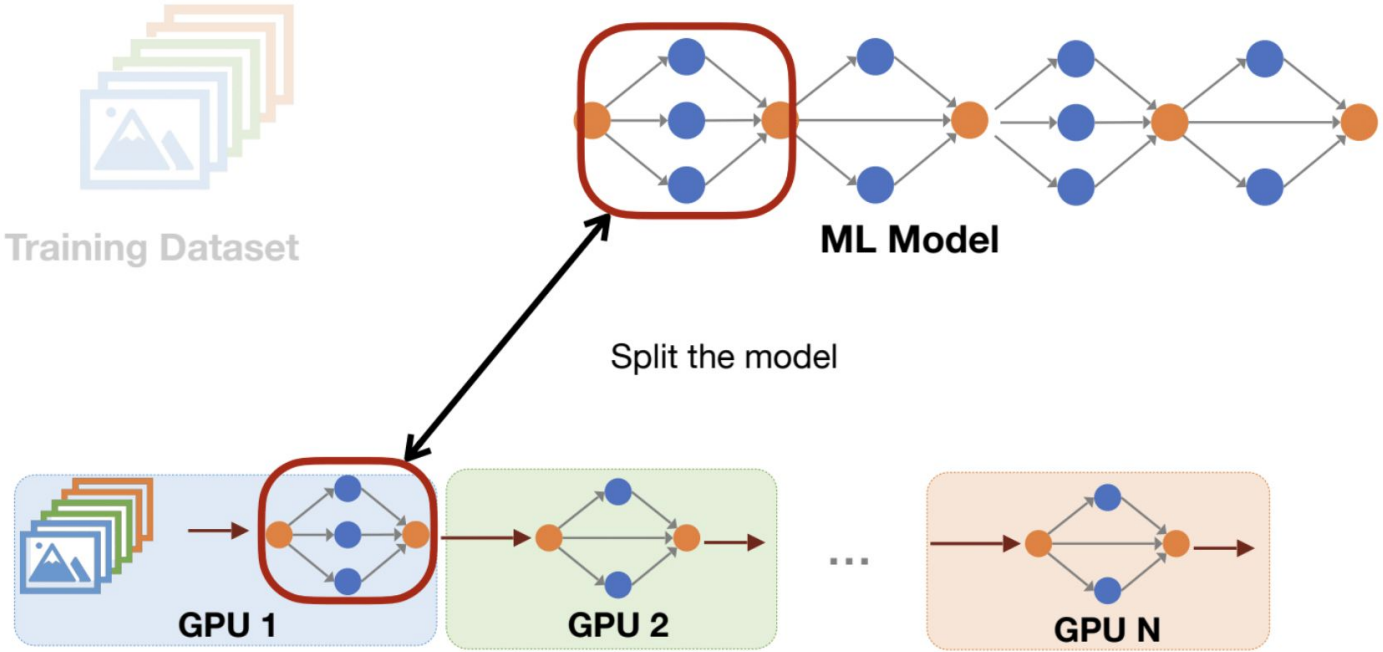
https://hanlab.mit.edu/courses/2024-fall-65940

Pipline Parallelism



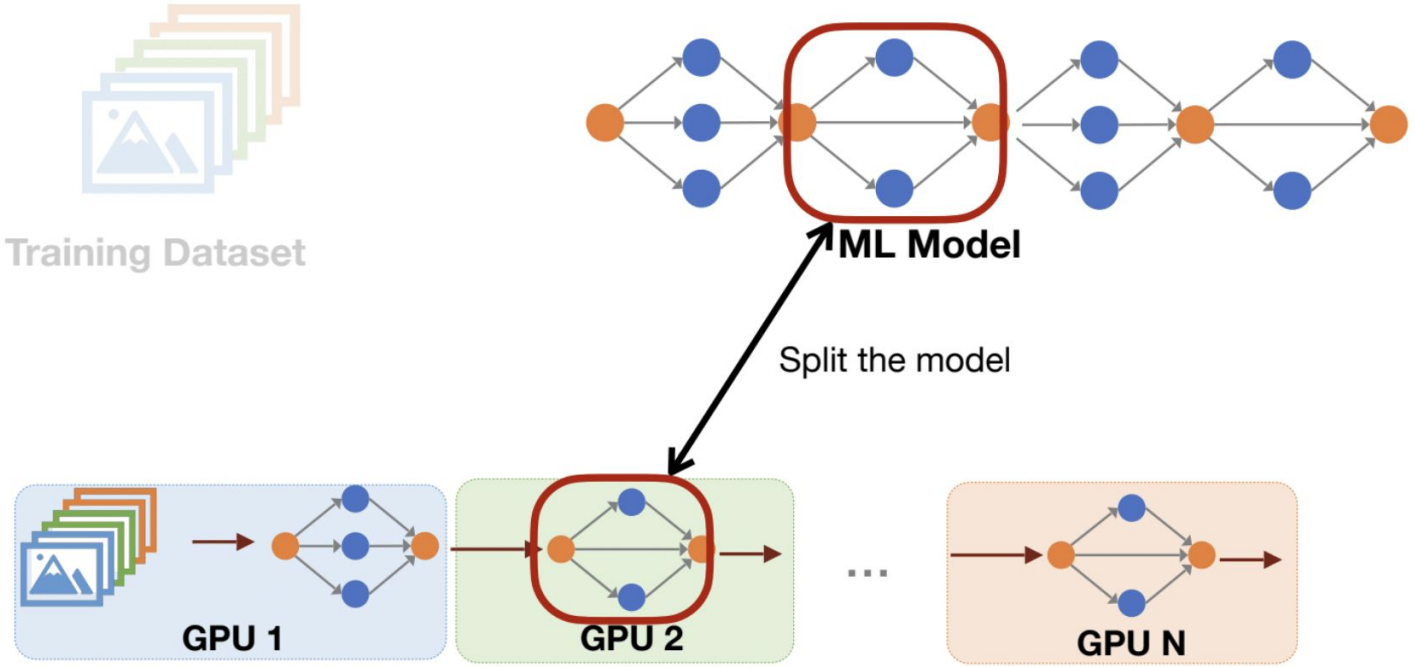
https://hanlab.mit.edu/courses/2024-fall-65940

Pipline Parallelism



https://hanlab.mit.edu/courses/2024-fall-65940

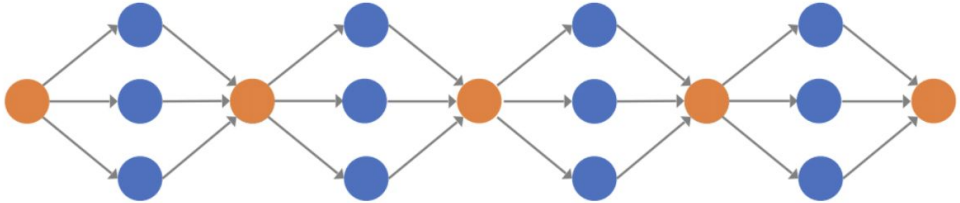
Pipline Parallelism



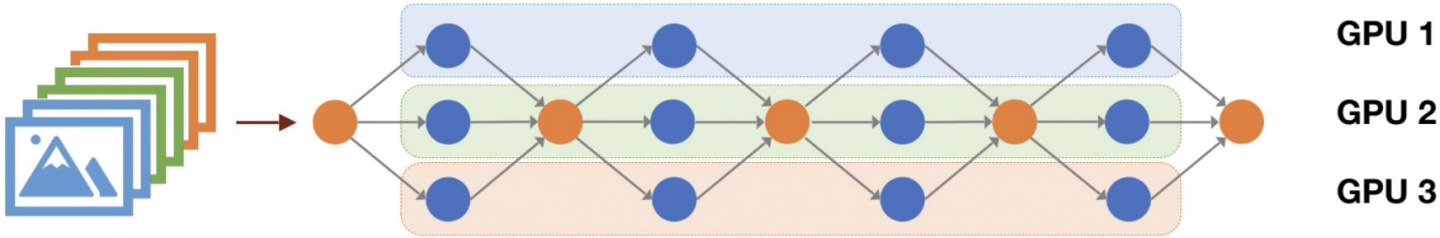
https://hanlab.mit.edu/courses/2024-fall-65940

Tensor Parallelism

Training Dataset

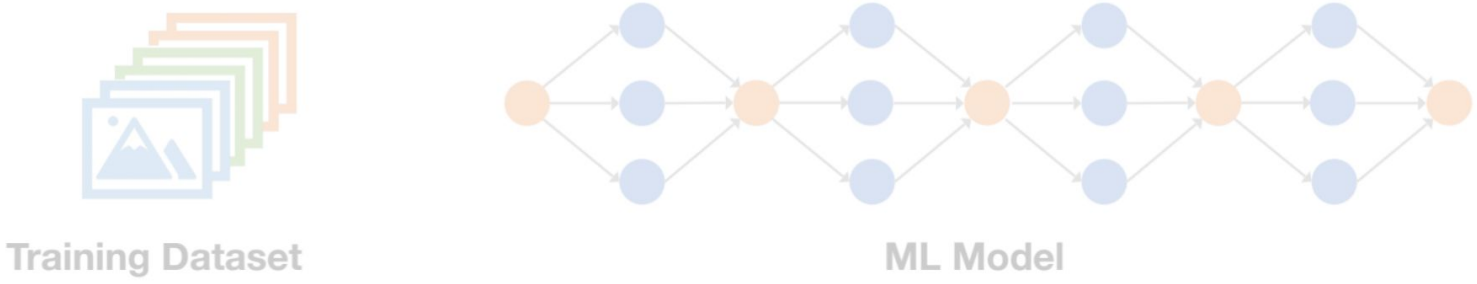


ML Model

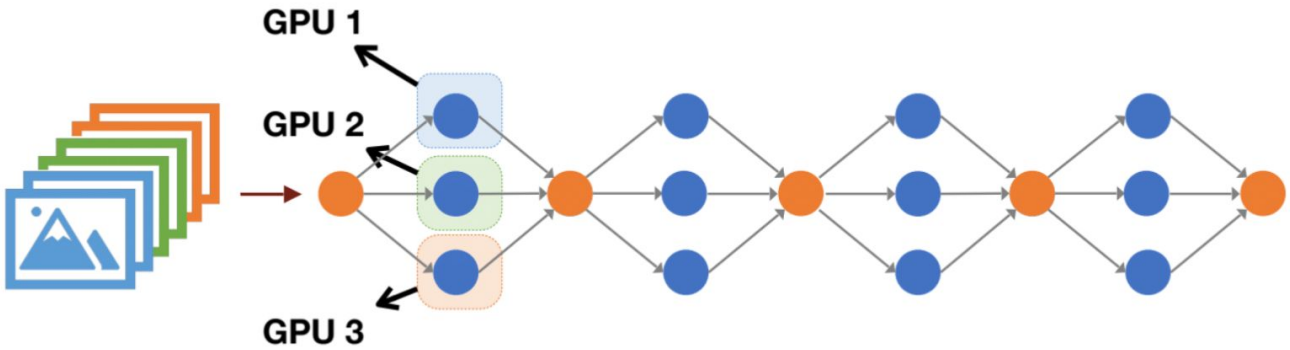


https://hanlab.mit.edu/courses/2024-fall-65940

Tensor Parallelism

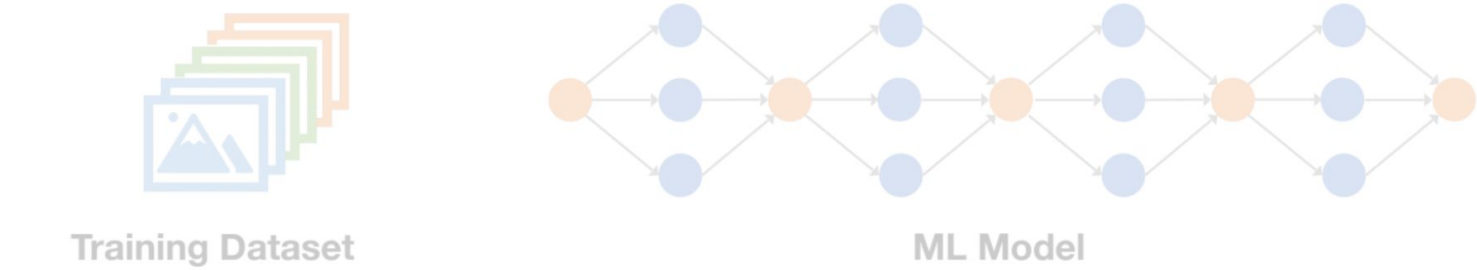


Split the activations then sync



https://hanlab.mit.edu/courses/2024-fall-65940

Tensor Parallelism



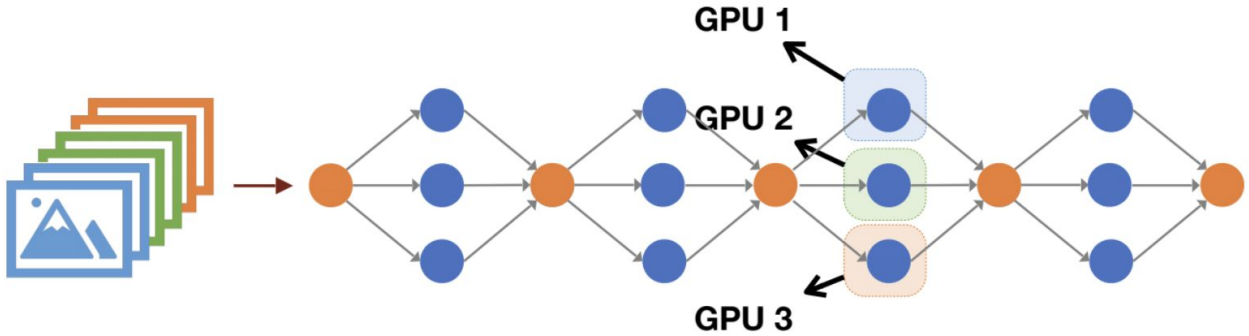
https://hanlab.mit.edu/courses/2024-fall-65940

Tensor Parallelism

Training Dataset

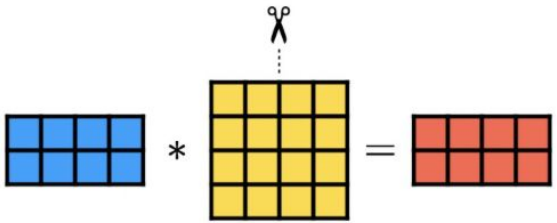
ML Model

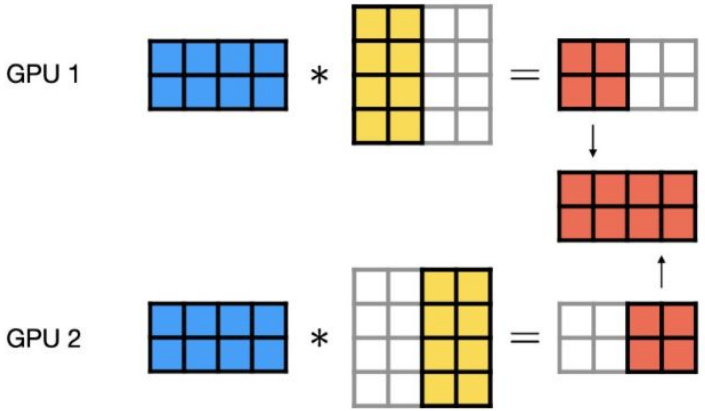
Split the activations then sync



https://hanlab.mit.edu/courses/2024-fall-65940

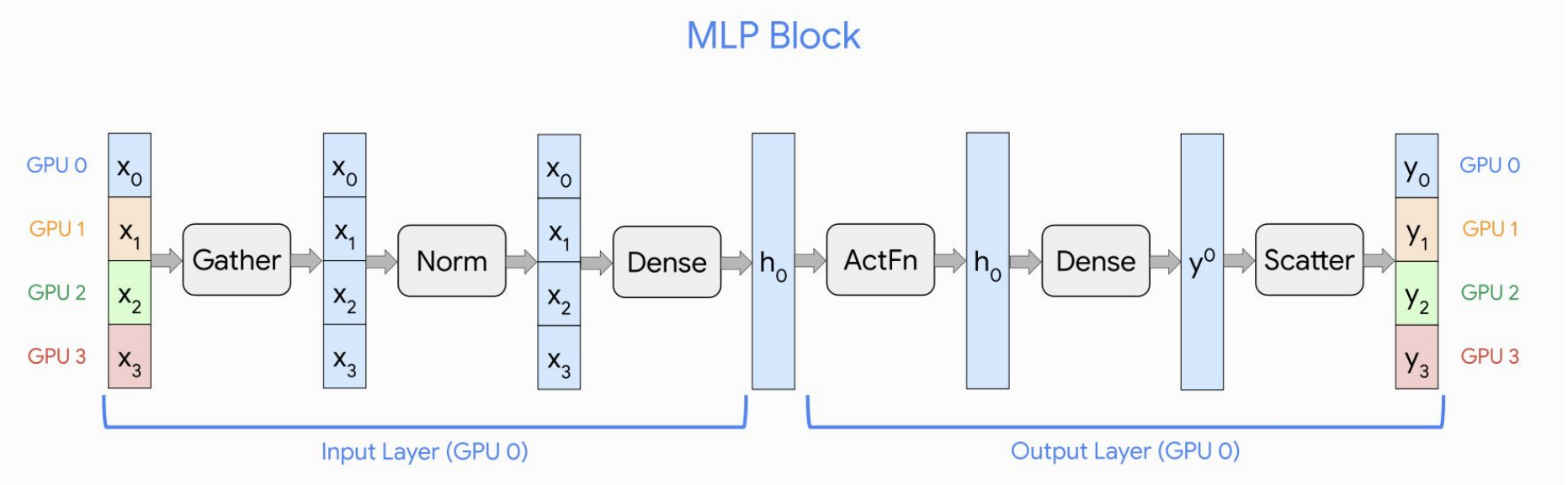
Tensor Parallelism





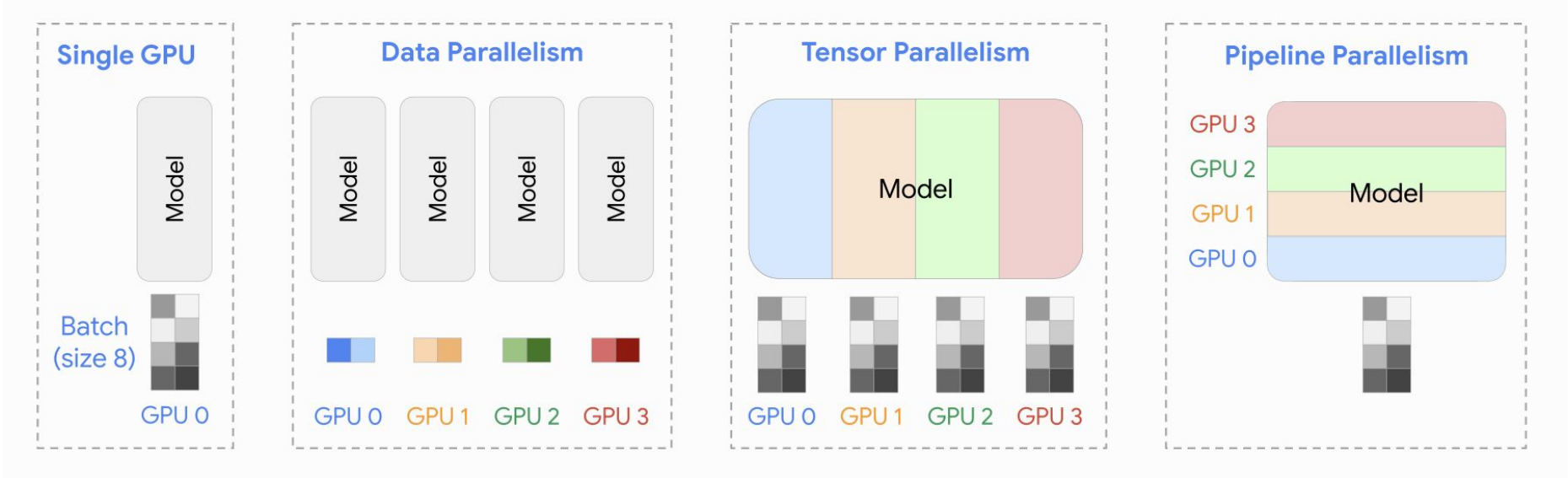
https://hanlab.mit.edu/courses/2024-fall-65940

Tensor Parallelism



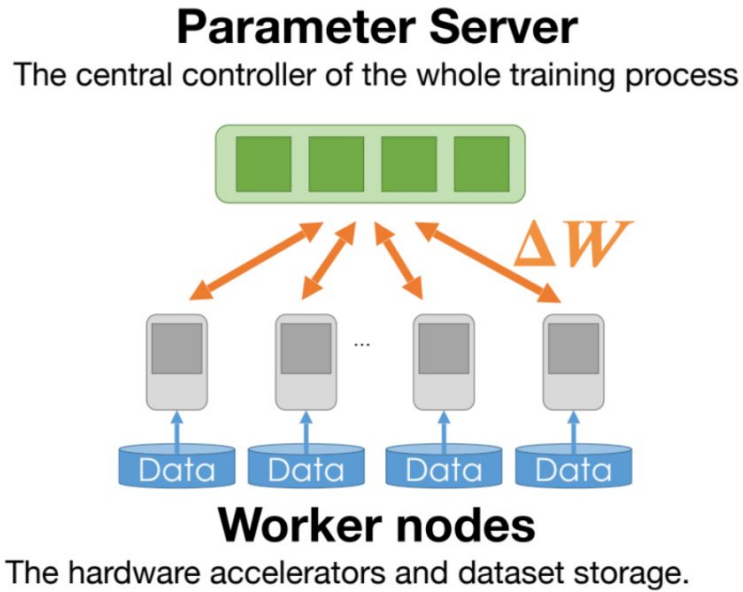
https://uvadlc-notebooks.readthedocs.io/en/latest/tutorial_notebooks/scaling/JAX/tensor_parallel_simple.html

Distributed training



https://uvadlc-notebooks.readthedocs.io/en/latest/tutorial_notebooks/scaling/JAX/tensor_parallel_simple.html

Data Parallelism



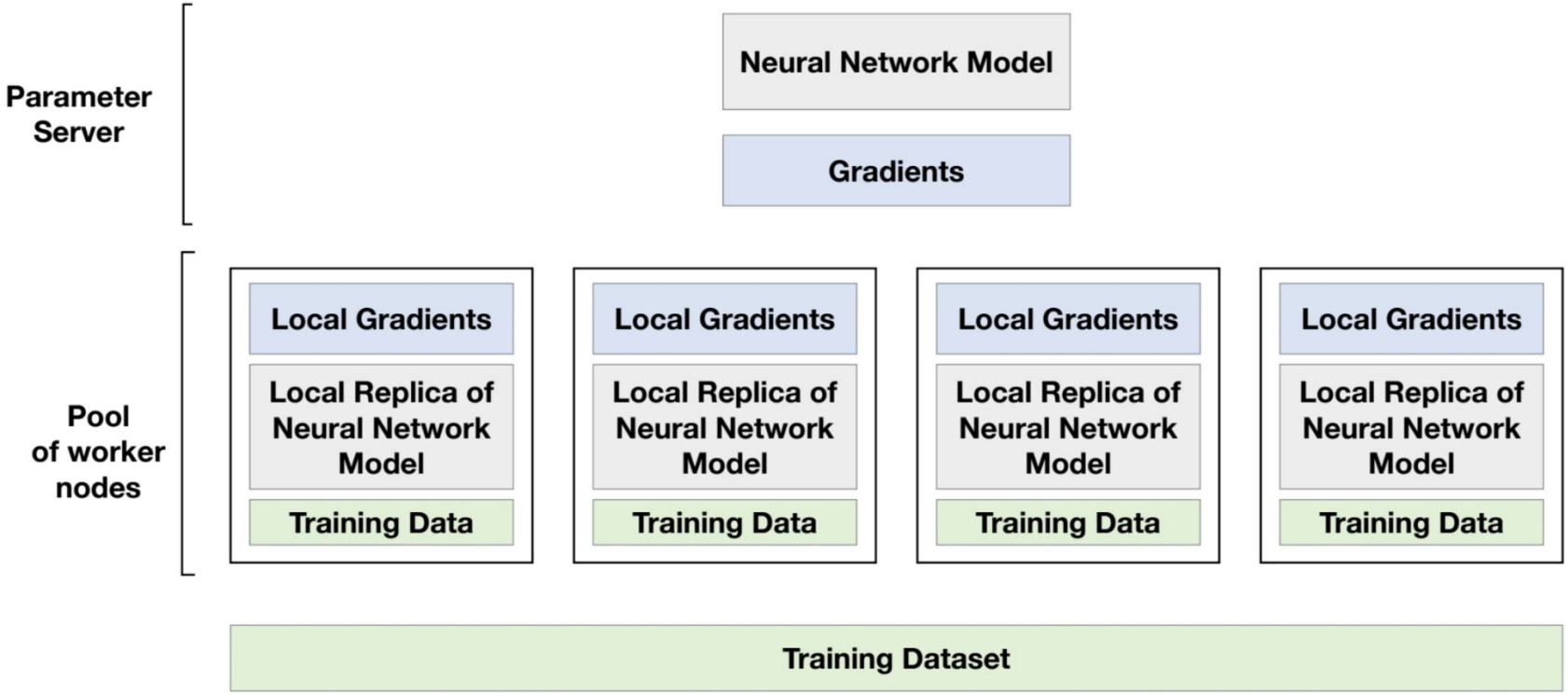
Two different roles in framework:

- Parameter Server: receive gradients from workers and send back the aggregated results
- Workers: compute gradients using splitted dataset and send to parameter server

Scaling Distributed Machine Learning with the Parameter Server. Mu Li et al. 2014
Figure credits from: Deep Gradient Compression. Lin et al. 2018

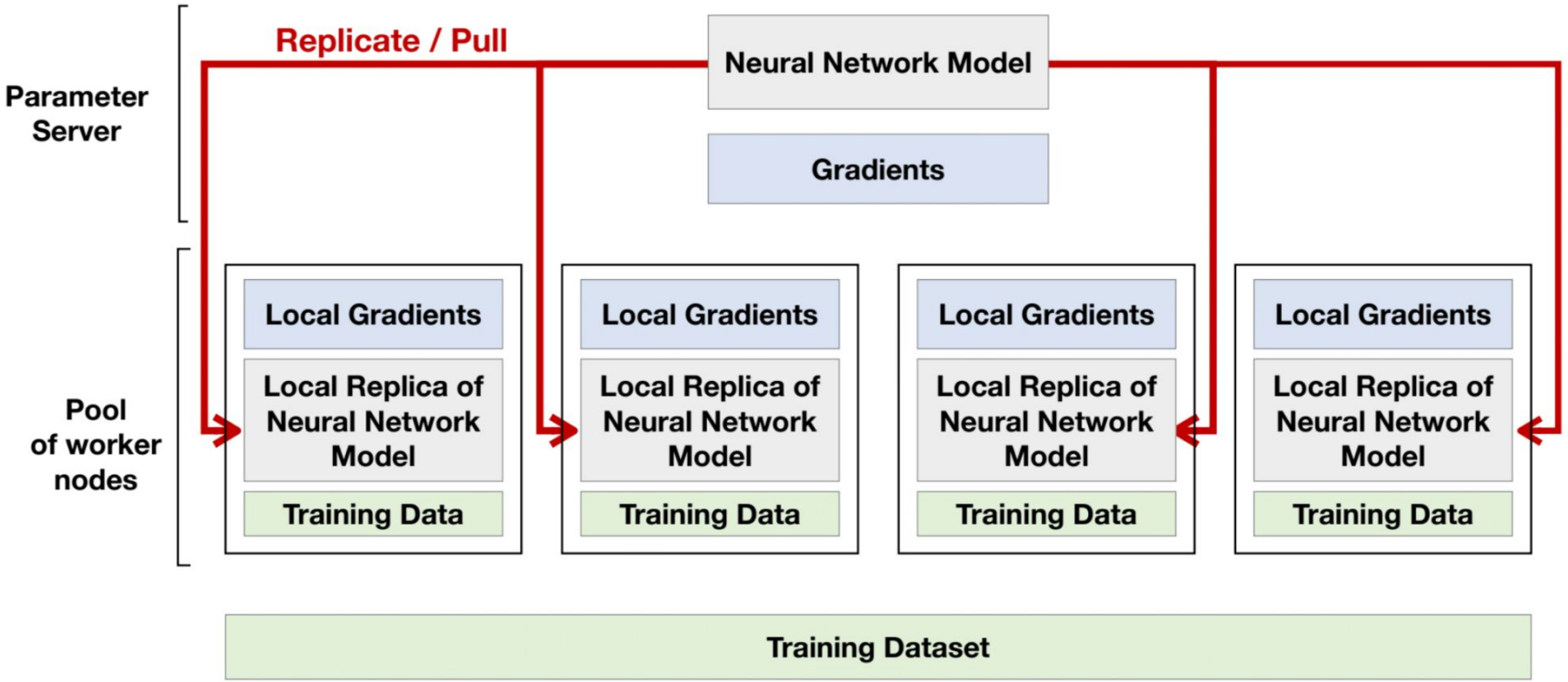
https://uvadlc-notebooks.readthedocs.io/en/latest/tutorial_notebooks/scaling/JAX/tensor_parallel_simple.html

Data Parallelism



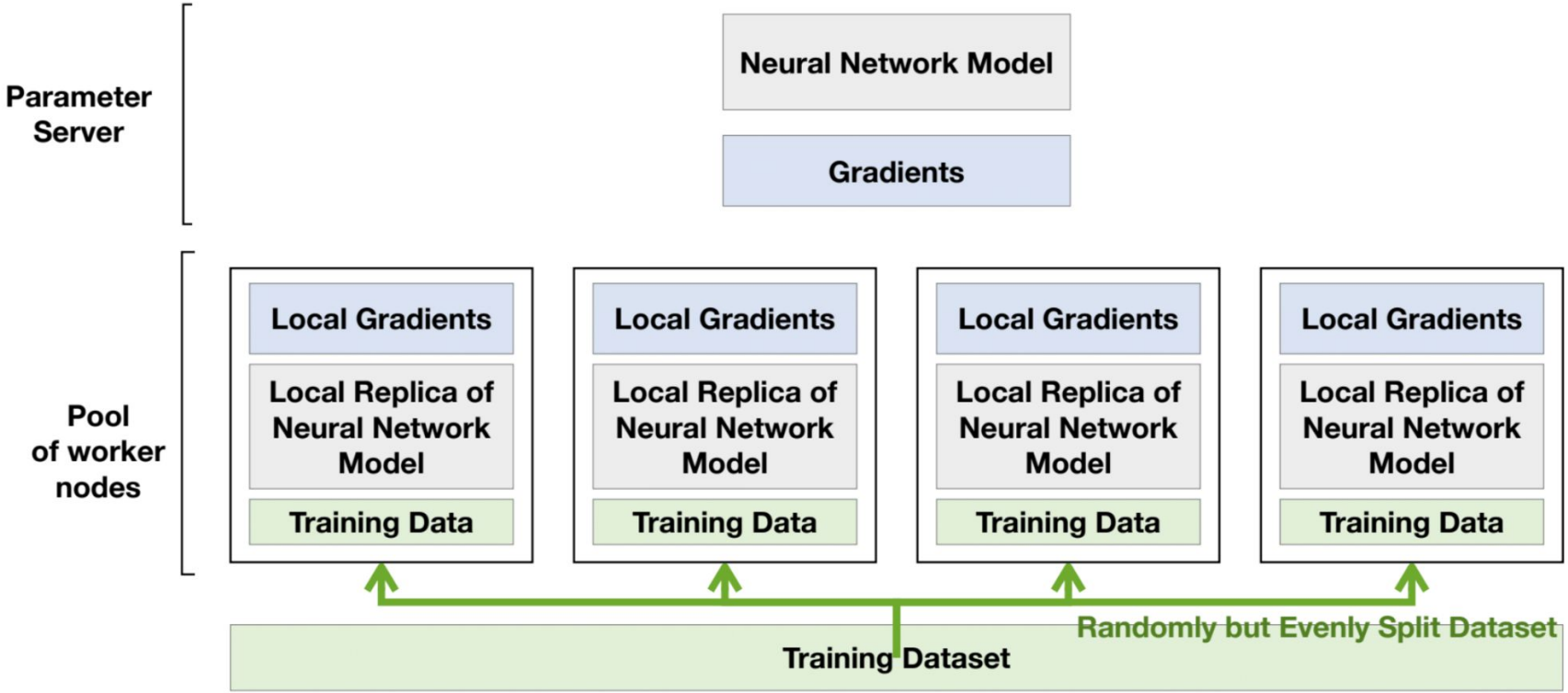
https://uvadlc-notebooks.readthedocs.io/en/latest/tutorial_notebooks/scaling/JAX/tensor_parallel_simple.html

Data Parallelism



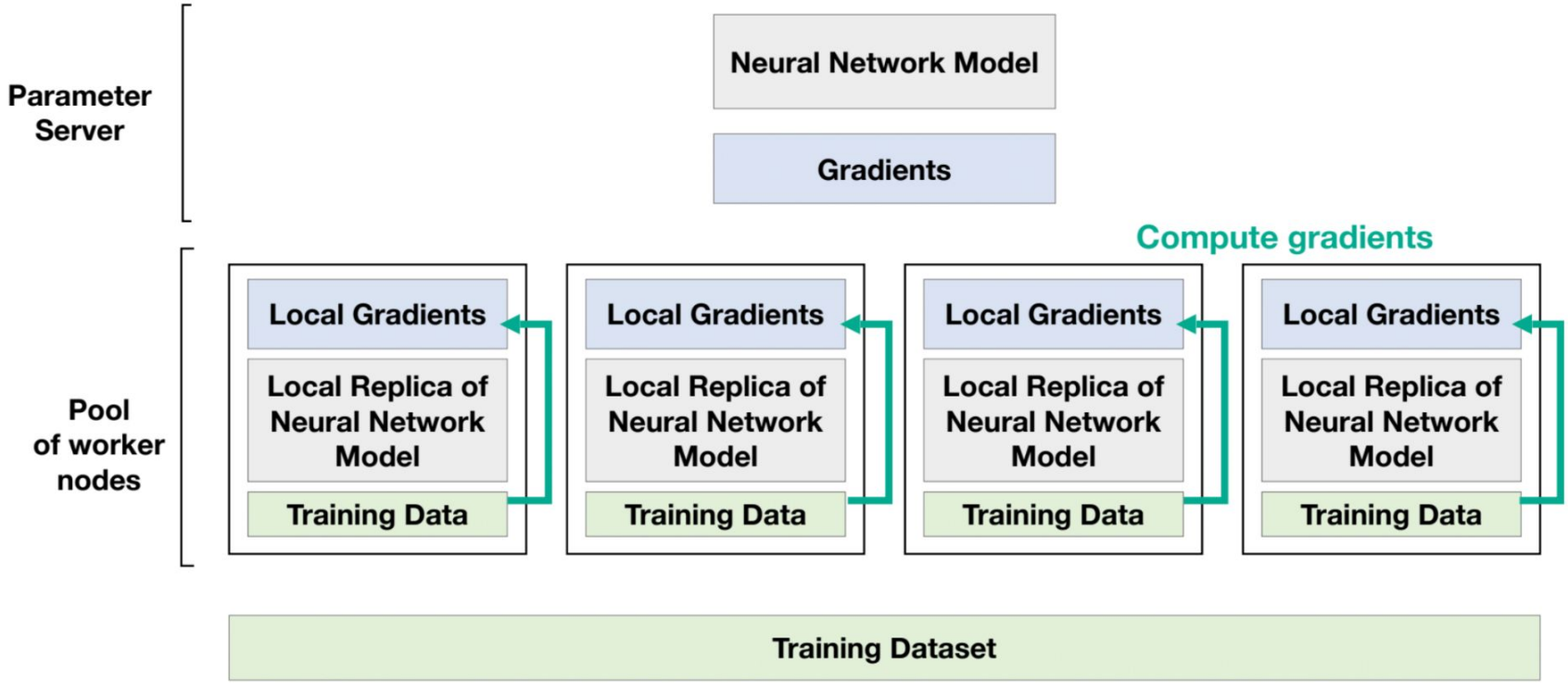
https://uvadlc-notebooks.readthedocs.io/en/latest/tutorial_notebooks/scaling/JAX/tensor_parallel_simple.html

Data Parallelism



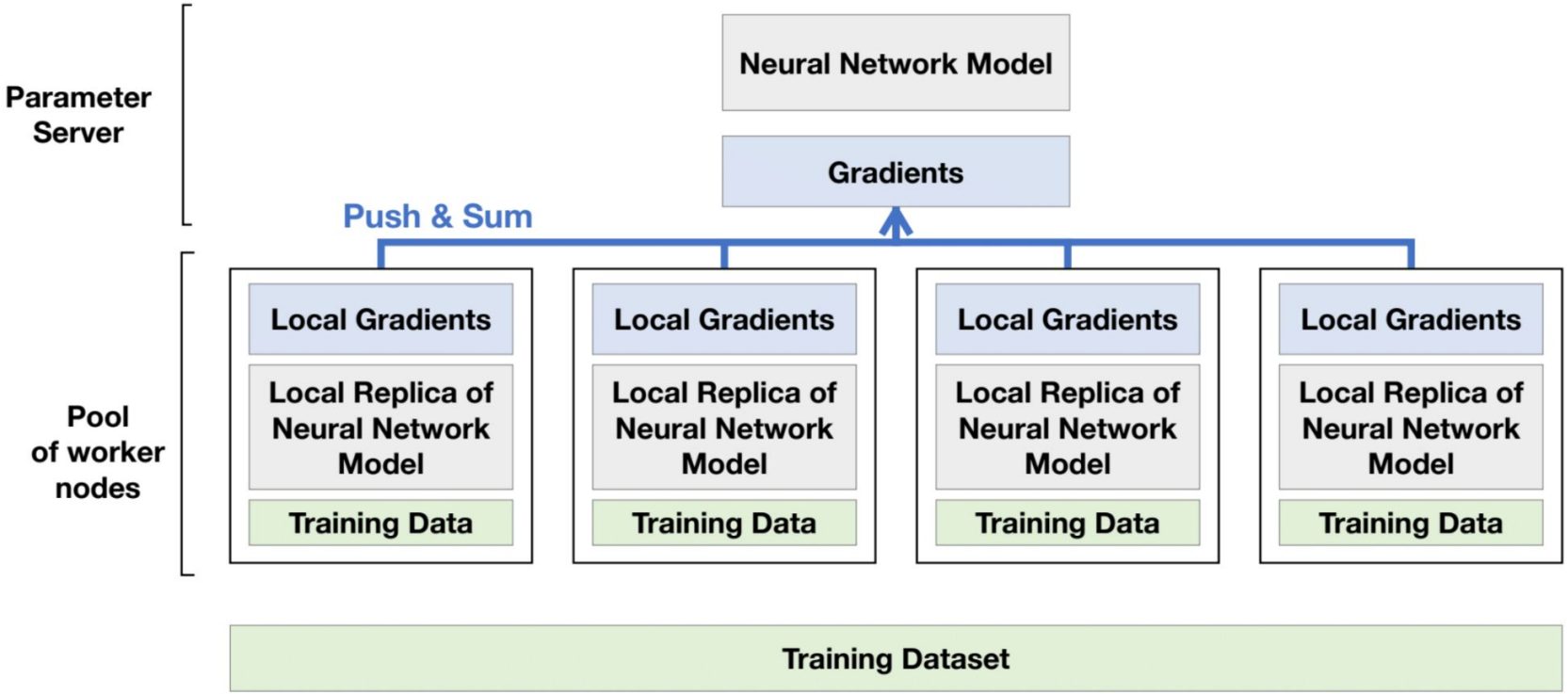
https://uvadlc-notebooks.readthedocs.io/en/latest/tutorial_notebooks/scaling/JAX/tensor_parallel_simple.html

Data Parallelism



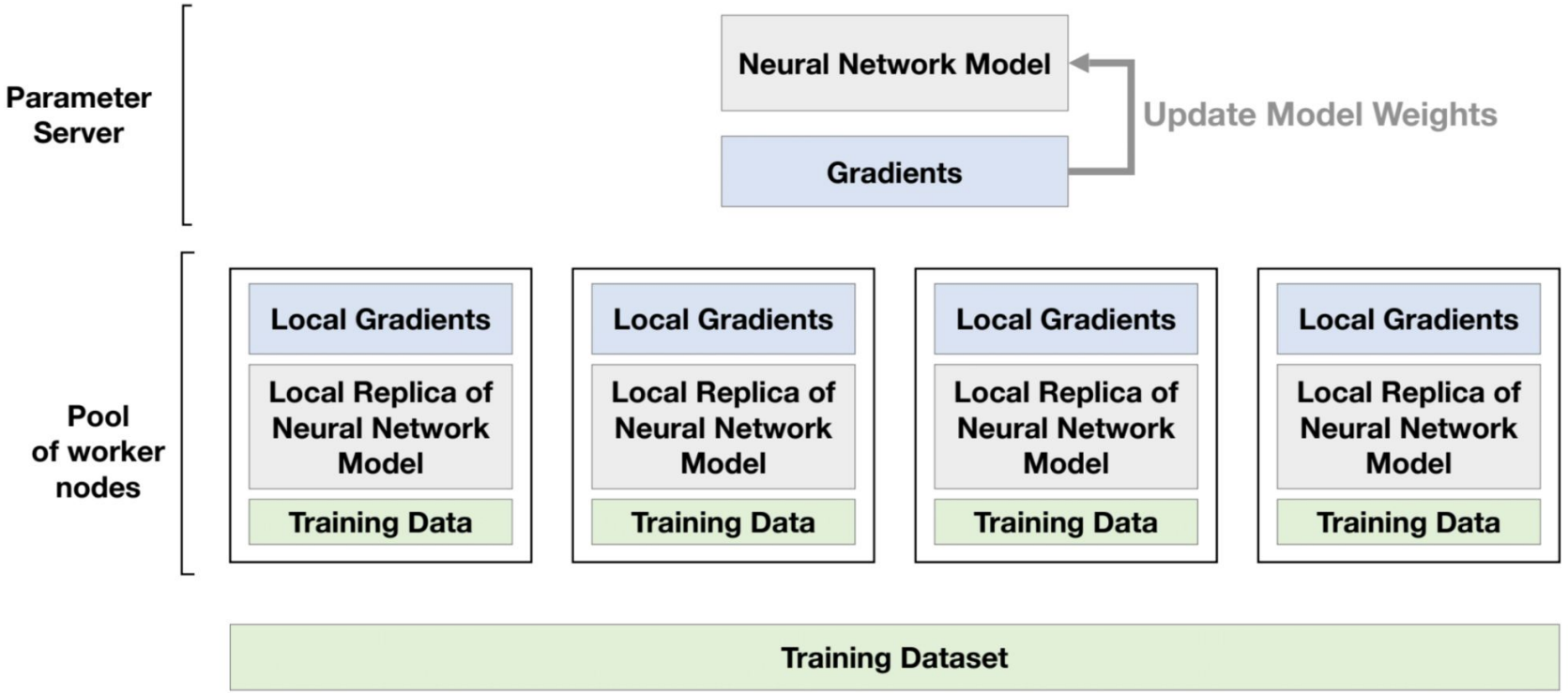
https://uvadlc-notebooks.readthedocs.io/en/latest/tutorial_notebooks/scaling/JAX/tensor_parallel_simple.html

Data Parallelism



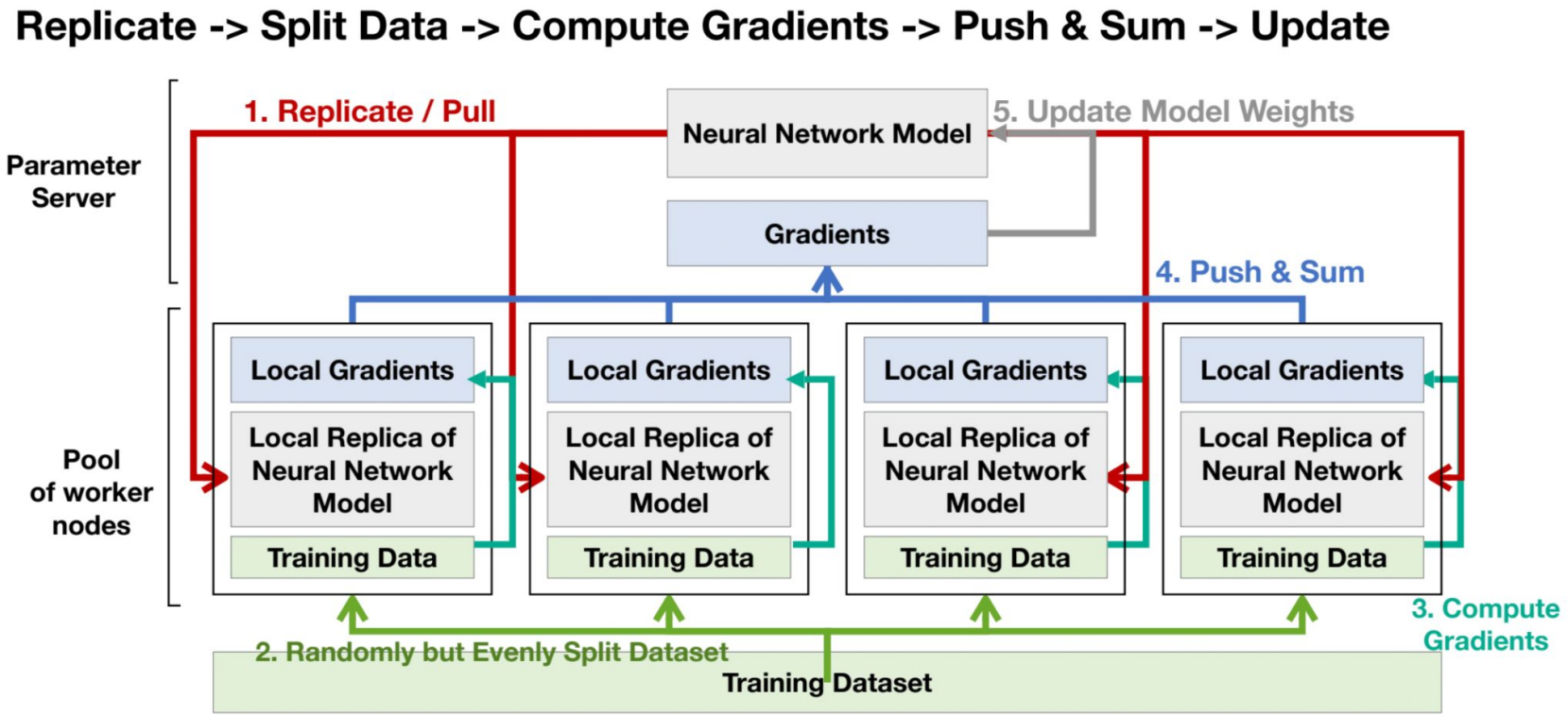
https://uvadlc-notebooks.readthedocs.io/en/latest/tutorial_notebooks/scaling/JAX/tensor_parallel_simple.html

Data Parallelism



https://uvadlc-notebooks.readthedocs.io/en/latest/tutorial_notebooks/scaling/JAX/tensor_parallel_simple.html

Data Parallelism



https://uvadlc-notebooks.readthedocs.io/en/latest/tutorial_notebooks/scaling/JAX/tensor_parallel_simple.html

Data Parallelism

Single Node Training:

For iteration i in 0 .. T,

1. Sample data from datasets $x_{(i)}, y_{(i)}$
2. Compute gradients $\nabla w_{(i)} = f(x_{(i)}, y_{(i)}; w_{(i)})$
3. Perform gradient step $w_{(i)} = w_{(i)} - \eta \nabla w_{(i)}$

Distributed Training:

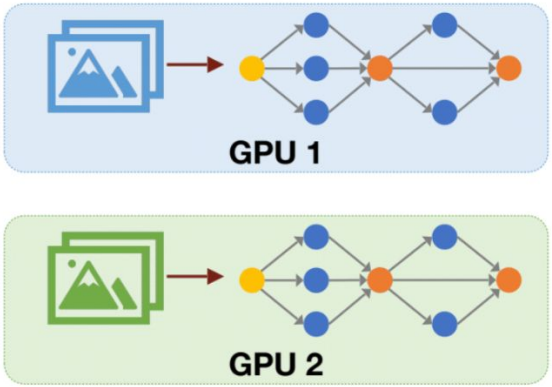
For iteration i in 0 .. T,

1. Replicate / pull gradients from parameter server
2. Sample data from datasets $x_{(i,j)}, y_{(i,j)}$
3. Compute gradients $\nabla w_{(i,j)} = f(x_{(i,j)}, y_{(i,j)}; w_{(i,j)})$
4. Push $\nabla w_{(i,j)}$ to parameter server and sum
5. Perform gradient step $w_{(i,j)} = w_{(i,j)} - \eta \overline{\nabla w_{(i)}}$ at parameter server.

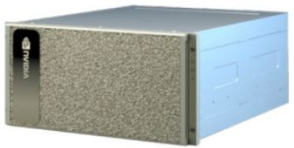
two synchronization steps

https://uvadlc-notebooks.readthedocs.io/en/latest/tutorial_notebooks/scaling/JAX/tensor_parallel_simple.html

ZeRO-1/2/3



If we train a super-large model (e.g., GPT-3 175B):



175B * 2 Bytes (fp16)
= 350GB Memory

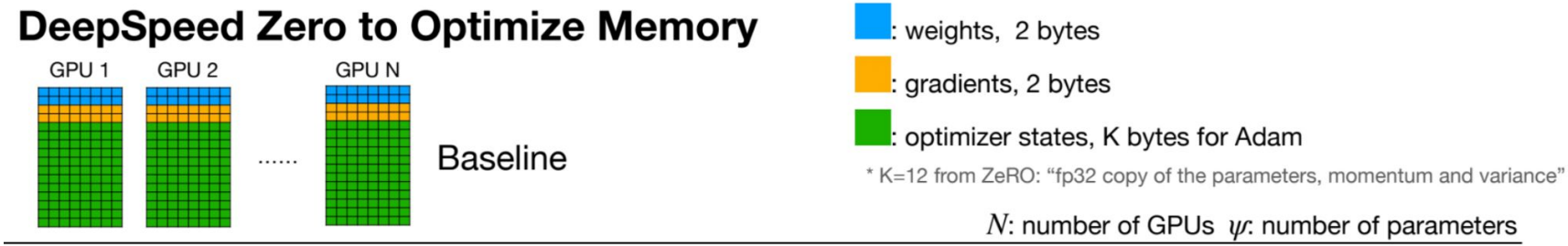
Nvidia A100 80GB

Even the best GPU **CANNOT** fit the model weights into memory!

Further, training requires to store gradients and optimizer states.

https://uvadlc-notebooks.readthedocs.io/en/latest/tutorial_notebooks/scaling/JAX/tensor_parallel_simple.html

ZeRO-1/2/3



- Naive data parallelism has N copies of weights, gradients and optimizer states.
- This introduces redundancy and easily leads to OOM when training large models.

$$(\underbrace{2\text{bytes}}_{\text{weights}} + \underbrace{2\text{bytes}}_{\text{gradients}} + \underbrace{12\text{bytes}}_{\text{optim states}})\psi$$

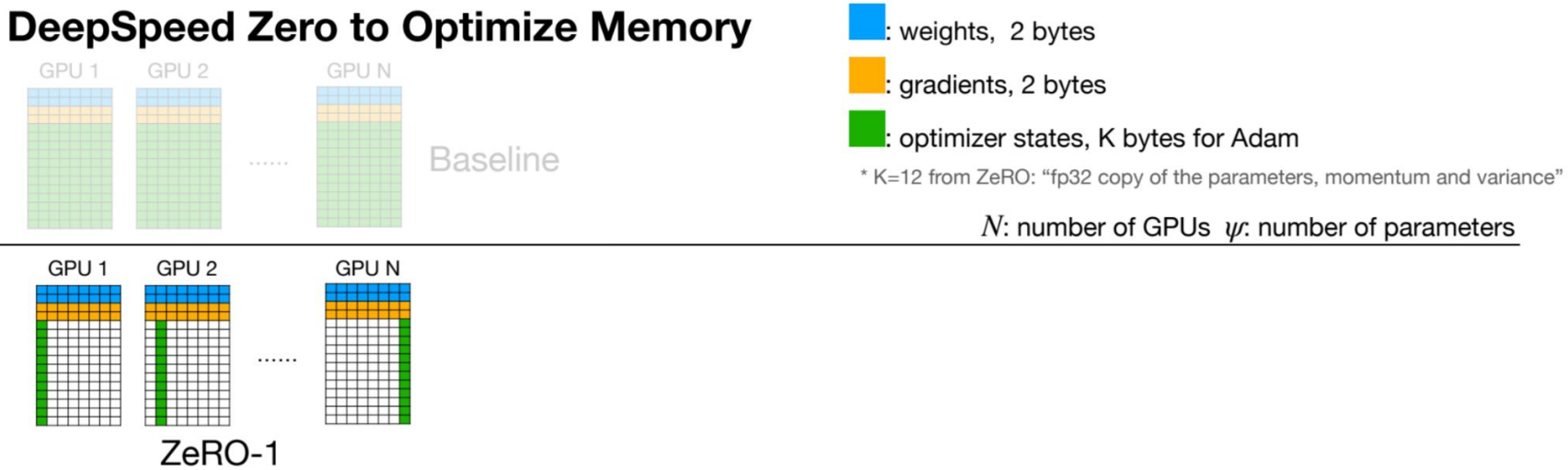
- Even for most most advanced A100/H100 with 80GB memory, the largest trainable model is

$$80\text{GB}/16\text{Bytes} = 5.0B$$

which is far away from current SOTA (175B).

https://uvadlc-notebooks.readthedocs.io/en/latest/tutorial_notebooks/scaling/JAX/tensor_parallel_simple.html

ZeRO-1/2/3



- Zero-1 proposes to partition / shard **optimizer states**.

Memory Consumption

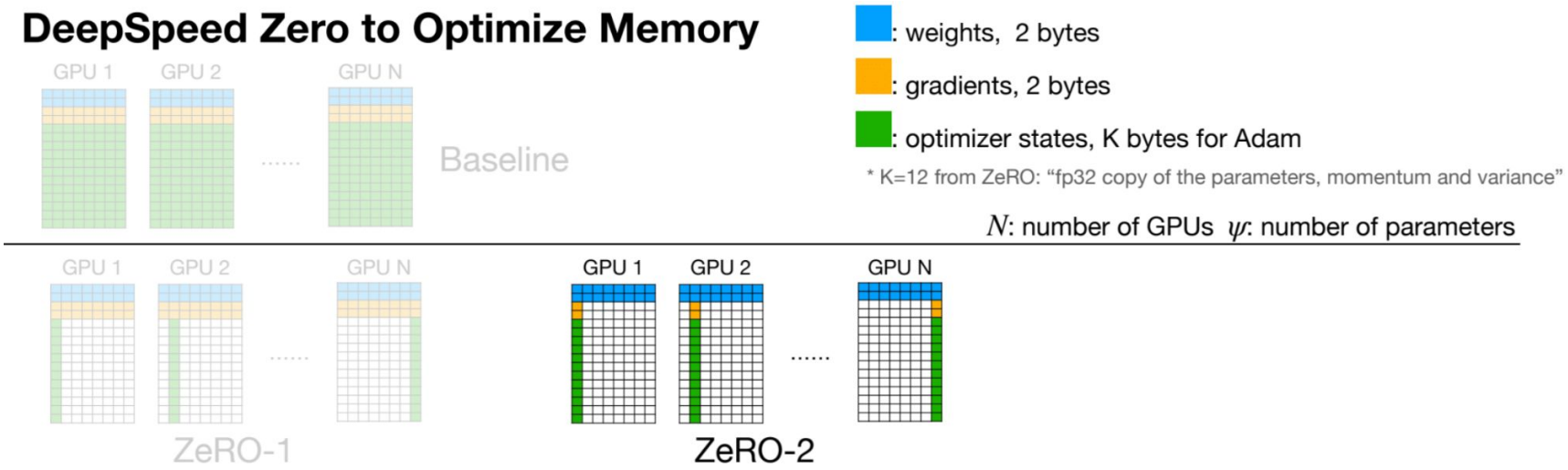
$$(\underbrace{2\text{bytes}}_{\text{weights}} + \underbrace{2\text{bytes}}_{\text{gradients}} + \underbrace{12/N\text{bytes}}_{\text{optim states}})\psi$$

Largest Model Size for Training (N=64)

$$80\text{GB}/4.2\text{Bytes} = 19B$$

https://uvadlc-notebooks.readthedocs.io/en/latest/tutorial_notebooks/scaling/JAX/tensor_parallel_simple.html

ZeRO-1/2/3



- Zero-2 proposes to partition / shard **optimizer states** and **gradients**.

Memory Consumption

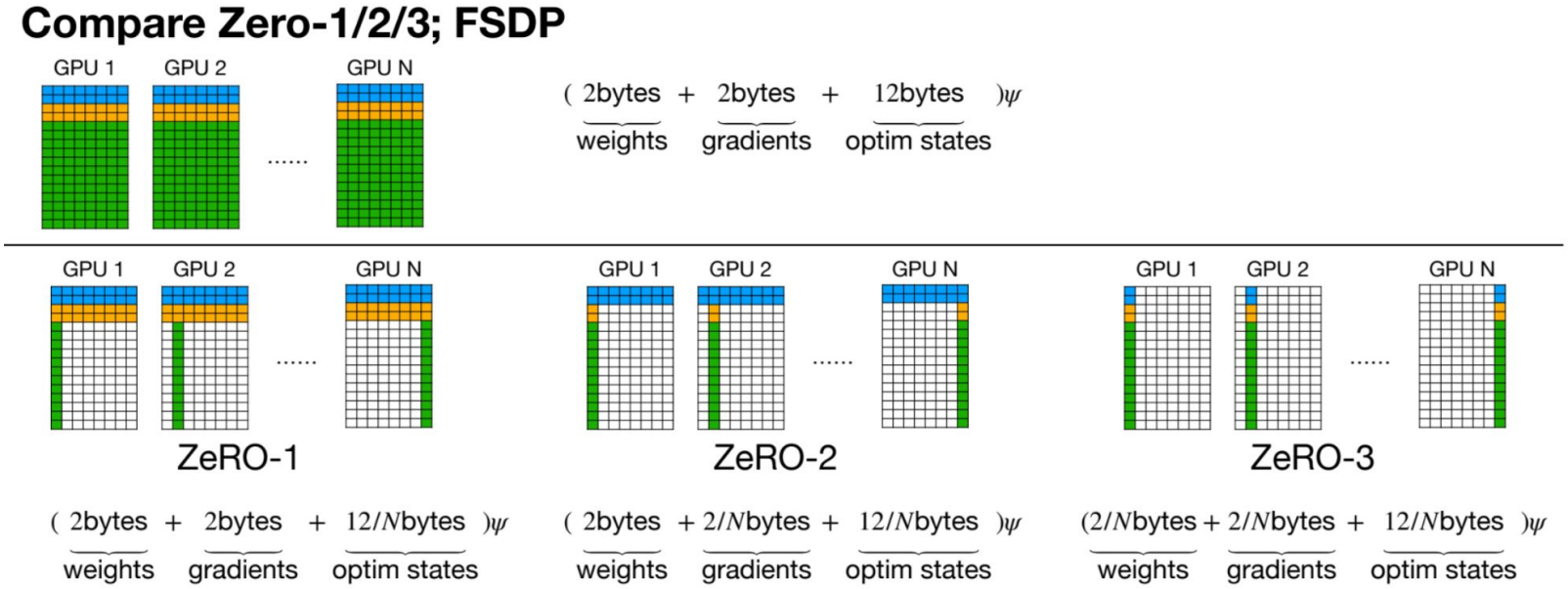
$$\Big(\underbrace{2\text{bytes}}_{\text{weights}} + \underbrace{2/N\text{bytes}}_{\text{gradients}} + \underbrace{12/N\text{bytes}}_{\text{optim states}} \Big)\psi$$

Largest Model Size for Training (N=64)

$$80\text{GB}/2.2\text{Bytes} = 36B$$

https://uvadlc-notebooks.readthedocs.io/en/latest/tutorial_notebooks/scaling/JAX/tensor_parallel_simple.html

ZeRO-1/2/3

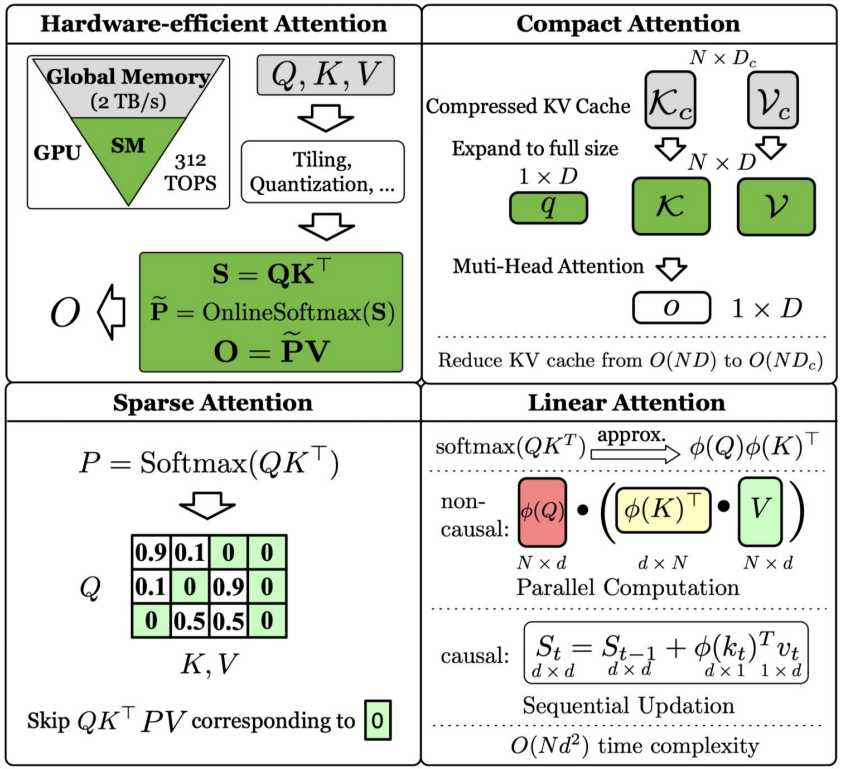


In PyTorch, ZeRO-3 is implemented via `FullyShardedDataParallel`, known as FSDP.

https://uvadlc-notebooks.readthedocs.io/en/latest/tutorial_notebooks/scaling/JAX/tensor_parallel_simple.html

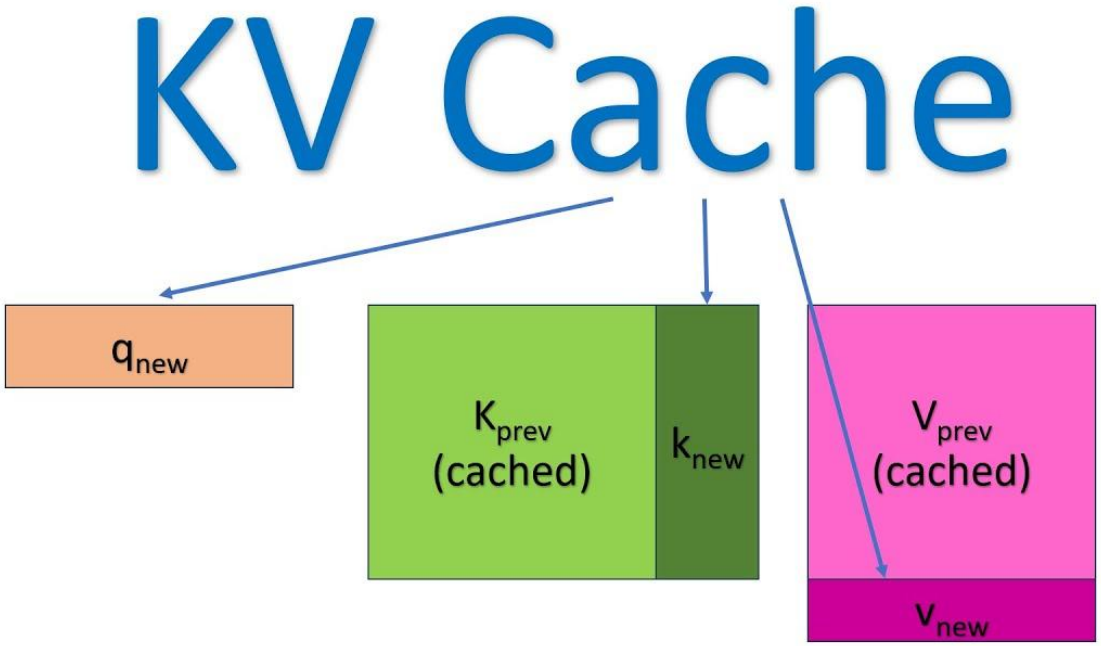
Efficient Attention

Efficient Attention



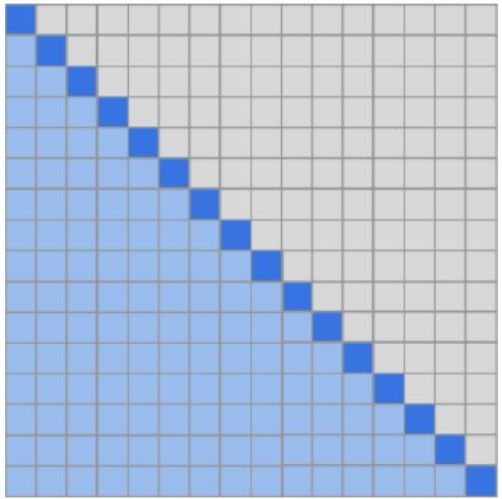
https://attention-survey.github.io/

KV-cache

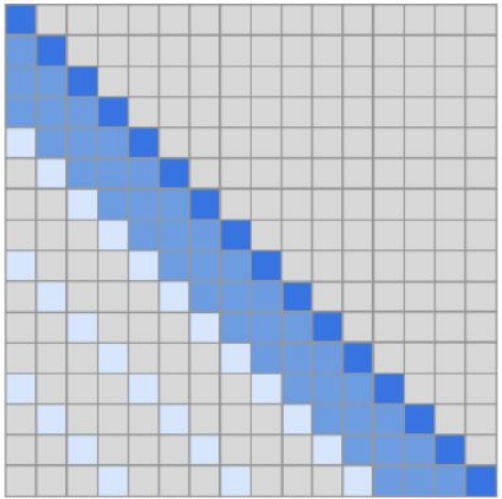


https://attention-survey.github.io/

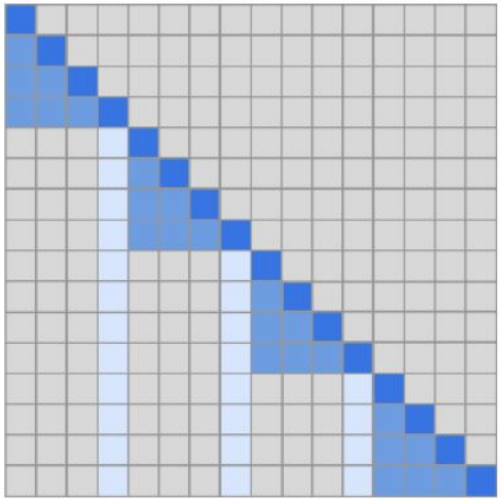
Attention matrix



(a) Transformer



(b) Sparse Transformer (strided)



(c) Sparse Transformer (fixed)

https://attention-survey.github.io/